QTree: An Accurate and Efficient Cardinality Estimator by Learning from Data and Queries via Access Affinity

Jiawei Liu*, Ju Fan*, Chao Zhang*, Tongyu Liu*, Kai Zeng§, Jiannan Wang†, Quehuan Liu*, Zhi Xu§, Nan Tang‡
*Renmin University of China, §Huawei Technologies,†Tsinghua University,‡HKUST(GZ)

***Abstract*—Cardinality estimation is a fundamental component in database systems, crucial for generating efficient execution plans. Despite advancements in learning-based cardinality estimation, existing methods may struggle to simultaneously optimize the key criteria: estimation accuracy, inference time, and storage overhead, limiting their practical applicability in real-world database environments. This paper introduces `QTree`, a tree-structured model integrating both data distribution and query workload via access affinity among columns. `QTree` achieves high estimation accuracy, low inference time and reduced storage overhead through two new node types: `QProduct` and `QSplit`, which enable further column partitioning based on access affinity from query access patterns. This paper studies the research challenges of developing efficient algorithms for the offline construction and online computation of `QTree`. And we also propose a novel join-free multi-table cardinality estimation method by extending `QTree`. We conduct extensive experiments to evaluate `QTree` on multiple single-table and multi-table datasets. The experimental results have demonstrated that `QTree` achieves superior and robust performance on the three key criteria, compared with state-of-the-art approaches. Our code is open-source at https://github.com/rucjrliu/QTreeCode.**

***Index Terms*—cardinality estimation, learning, access affinity**

I. Introduction

Cardinality estimation (CardEst), which estimates the result size of an SQL query on a relational database, is a fundamental component of query optimization in database management systems (DBMSs). Traditional CardEst methods [1]–[7] rely on simplifying assumptions, such as column independence, often leading to substantial estimation errors. To overcome these limitations, learning-based CardEst models [8]–[14] have emerged as state-of-the-art solutions, significantly improving accuracy by capturing data distributions and query patterns.

Despite the advancements in learning-based CardEst models, deploying them in real-world DBMS systems still requires balancing three key criteria [14]–[18]: *estimation accuracy*, *inference time*, and *storage overhead*. Data-driven approaches [10], [11], [13], [14] leverage probabilistic models, such as tree-structured Sum-Product Networks (SPN) [12]–[14] and Deep Auto-Regressive (DAR) [10], [11], to capture the joint distribution of all columns of the relational data. While these methods typically achieve high estimation accuracy, they suffer from high inference time and substantial storage overheads, particularly when handling complex data distributions. Query-driven methods [8], [9] train regression models that directly map queries to their cardinalities based on a set of training queries, bypassing the need to model data distributions. Although these methods are efficient and lightweight, they struggle with accuracy, particularly when encountering queries that significantly deviate from those in the training set.

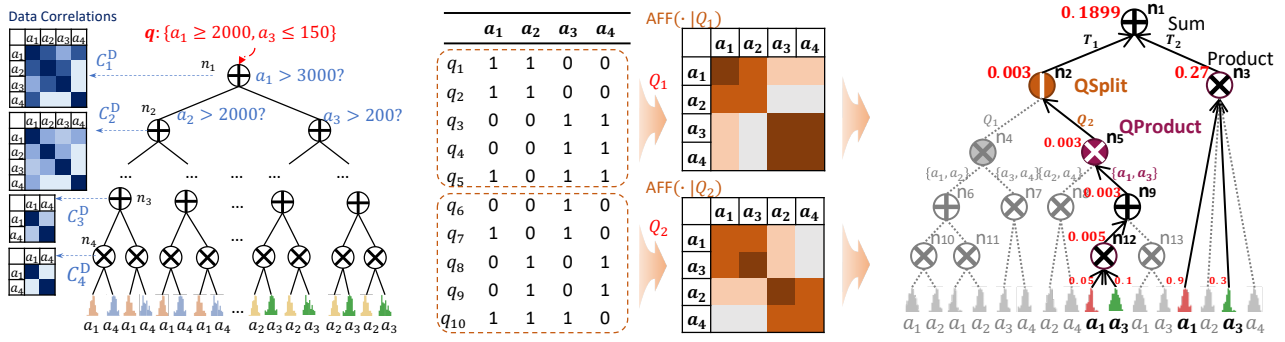
Given the strengths and limitations of existing methods, a promising direction is to develop a *hybrid* model learning from both data and queries, aiming to achieve the three key criteria: *high estimation accuracy*, *low inference time*, and *lightweight storage overhead*. Such an approach can overcome the weaknesses of data-driven or query-driven CardEst models by leveraging complementary information from both sources.

Our Proposals. In this paper, we propose a tree-structured model `QTree` to learn from both data and queries. As shown in Figure 1(a), traditional SPN models [13], [14] recursively partition columns (*i.e.,* the `Product` node) and rows (*i.e.,* the `Sum` node) into *local subsets*, making it easier to compute the joint probability distribution from these local distributions. However, they often suffer from *high inference time* and *large model size* when columns in a database are highly correlated. This issue arises because many intermediate nodes (*e.g.,*`Sum` nodes) must be introduced to ensure that the columns in partitioned subsets are treated as independent.

To address this problem, `QTree` partitions column set by incorporating query workload information. The high-level idea of `QTree` stems from an observation in many real-world workloads: queries often exhibit specific access patterns, affecting **access affinity** among columns, which can be leveraged to enhance both the efficiency and accuracy of CardEst. For instance, in STATS-CEB, a benchmark workload designed to reflect real-world queries in *Stats Stack Exchange*, 55 queries apply filter predicates to the six-column 'votes' table. Of these queries, 16.36% filter on both 'BountyAmount' and 'CreationDate', 9.09% on both 'CreationDate' and 'VoteTypeId', 49.09% on 'CreationDate' alone, 10.91% on 'BountyAmount' alone, and 14.55% on 'VoteTypeId' alone, with no other patterns.

By integrating query workload information, `QTree` can jointly partition columns based on both data correlations and query access patterns, thereby reducing model size and improving inference efficiency without sacrificing estimation accuracy. Table I presents a comparison between `QTree` and existing approaches by the three key criteria mentioned above.

Example 1. *We consider the CardEst task for an example table T with highly correlated columns a_1, a_2, a_3, a_4, as illustrated in Figure 1(a). SPN partitions T into different row*



(a) Traditional SPN Model. (b) Partitioning Columns based on Query Patterns. (c) QTree Model and CardEst Computation.

Fig. 1. High-level idea of QTree.

TABLE I
CARDINALITY ESTIMATION METHODS COMPARISON

Method	Category	Accurate Estimation	Fast Inference	Lightweight Storage
Postgres	Traditional	☆	★★★	★★★
MHist	Traditional	☆	☆	★
Sampling	Traditional	★★	★★	★★★
MSCN [8]	Query-Driven	★★	★★★	★
Naru [10]	Data-Driven	★★★	★	★
DeepDB [13]	Data-Driven	★★	★★	★★
FLAT [14]	Data-Driven	★★★	★	☆
UAE [19]	Hybrid	★★★	★	★
Ours	**Hybrid**	★★★	★★★	★★★

subsets via Sum *nodes (*e.g., *node n_1, which partitions rows based on whether $a_1 > 3000$) to reduce column correlations within each subset. However, as depicted in the figure, when columns exhibit high correlations, the SPN requires numerous* Sum *nodes to break down the joint distribution into local distributions over individual columns. This leads to a substantial increase in model size. Moreover, when processing a query q, the inference procedure must traverse a large number of these nodes, significantly increasing inference time.*

As illustrated in Figure 1(b), QTree *leverages the access patterns of workload Q,* i.e., *how often certain columns are accessed together by the same queries. Within subset Q_1 of Q, we observe that columns a_1 and a_2 are frequently accessed together by queries q_1 and q_2, while a_3 and a_4 are jointly accessed by queries q_3 and q_4. Based on this pattern, we partition the columns into $\{a_1, a_2\}$ and $\{a_3, a_4\}$ for queries in Q_1, even if their data remain highly correlated. Similarly, for queries in subset Q_2, the columns can be partitioned into $\{a_1, a_3\}$ and $\{a_2, a_4\}$. These query-aware column partitions allow* QTree *to construct a more compact tree-structured model, reducing model size and inference time while maintaining accuracy.*

As shown in Figure 1(c), we formalize QTree as a tree-structured model that introduces two new node types, QProduct and QSplit, to enable workload-aware column partitioning in addition to row and column partitioning based on data correlations. Specifically, QProduct partitions columns based on query-specific access patterns (*e.g.,* grouping frequently co-accessed columns together) within a given workload, while QSplit refines the workload itself into more granular subsets to capture workload variations. By partitioning columns based on both data correlations and query access patterns, QTree effectively reduces the number of intermediate nodes in the tree, which improves inference efficiency and reduces storage overhead while maintaining high estimation accuracy.

Key Challenges and Solutions. We study the technical challenges that naturally arise in our proposed QTree approach.

Offline QTree Construction. A key challenge in offline QTree construction is integrating query workload information into the data joint distribution. QTree must consider both data correlations and query co-access patterns, making the partitioning problem more complex. To address this, QTree develops efficient algorithms for two core problems, *i.e.,* QProduct for query-aware column partitioning and QSplit for workload partitioning.

Online QTree Computation. The online stage of QTree presents two key challenges. First, accurate CardEst computation with minimizing inference overhead is non-trivial, especially for queries with unseen access patterns. Second, as data distributions and query workloads evolve, QTree must maintain accuracy without frequent full retraining. To tackle these challenges, we develop an online inference algorithm, and introduce an incremental update mechanism that enables QTree to efficiently adapt to workload and data changes.

Multi-Table CardEst with QTree. Extending QTree to multi-table CardEst is challenging due to the complex distributions of join keys and the impact of base table query predicates. To address this, we introduce a join-free multi-table CardEst method based on statistics and single-table QTree models, maintaining high accuracy and efficient inference.

Contributions. Our contributions are summarized as follows.

(1) We propose QTree, a tree-structured CardEst model learning from both data and queries (Section III).

(2) We develop effective and efficient algorithms for offline QTree construction and online QTree computation (Sections IV and V). We propose QTree supporting multi-table CardEst (Section VI).

(3) We conduct a comprehensive experimental study on widely used CardEst benchmarks. Extensive results demonstrate that QTree achieves superior performance (Section VII).

II. Preliminaries

A. Problem Formalization

Data. This paper considers a relational table T with columns (attributes) $A = \{a_1, a_2, \ldots, a_{|A|}\}$ and tuples $T = \{t_1, t_2, \ldots, t_{|T|}\}$. We define the domain of column a_i as $[LB_i, UB_i]$, representing the lower and upper bounds of a_i.

Queries. Similar to existing works [13], [14], [19], this paper focuses on queries that consist of a *conjunction* of predicates, where each predicate over column a_i can be represented as $L_i \leq a_i \leq U_i$, with $LB_i \leq L_i \leq U_i \leq UB_i$. For ease of presentation, we assume each column has only one interval $[L_i, U_i]$, though this can be easily extended to the cases with multiple intervals per column.

This paper considers a *query workload* as a set of queries $Q = \{q_1, q_2, \ldots, q_{|Q|}\}$, typically extracted from query logs.

CardEst. Given a new query q, we define $\texttt{Card}(q, T)$ as the *cardinality* of q over table T, *i.e.,* the number of tuples in T that satisfy the query conditions. The goal of *cardinality estimation* is to compute an estimate $\widehat{\texttt{Card}}(q, T)$ that approximates $\texttt{Card}(q, T)$ efficiently and accurately without executing q on T. This problem can be addressed by modeling the *joint probability distribution* $P_T(A)$, where $\forall a_i \in A$ is treated as a random variable. Given a query q with predicates $a_i \in [L_i, U_i]_{i=1}^m$, the cardinality is estimated as $\widehat{\texttt{Card}}(q, T) = |T| \cdot \sum_{a_1 \in [L_1, U_1]} \cdots \sum_{a_m \in [L_m, U_m]} P_T(a_1, \ldots, a_m)$, where the summation iterates over all possible values within the query's range constraints.

III. An Overview of QTree

We propose QTree, a tree-structured CardEst model incorporating query workload information to partition columns based on *access affinity* from their *access patterns*. We formalize QTree as a tree containing five node types: QProduct, QSplit, Product, Sum, Leaf, as shown in Figure 1(c). Formally, each node n in QTree is represented as a 4-tuple $(A_{\texttt{n}}, T_{\texttt{n}}, Q_{\texttt{n}}, O_{\texttt{n}})$, where $A_{\texttt{n}}$ denotes the column set, $T_{\texttt{n}}$ the corresponding table, and $Q_{\texttt{n}}$ the associated query workload. Each node captures the joint probability distribution $P_{T_{\texttt{n}}}(A_{\texttt{n}})$ conditioned on the queries in $Q_{\texttt{n}}$. Moreover, the node type $O_{\texttt{n}}$ represents how the joint probability of node n is estimated from its child nodes, which is described as follows.

QProduct. A QProduct node, such as $\texttt{n}_5$ in Figure 1(c), partitions its column set $A_{\texttt{n}}$ into a set of disjoint subsets $\mathcal{A} = \{A_1, \ldots, A_m\}$, ensuring that columns in different subsets are infrequently co-accessed by queries in $Q_{\texttt{n}}$. For each subset A_i, a corresponding child node is created as $\texttt{n.child}_i = (A_i, T_{\texttt{n}}[A_i], Q_{\texttt{n}}[A_i], O_i)$. The joint probability distribution $P_{T_{\texttt{n}}}(A_{\texttt{n}})$ at node n is computed as $P_{T_{\texttt{n}}}(A_{\texttt{n}}) = \prod_{i=1}^m P_{T_{\texttt{n}}}(A_i)$.

QSplit. A QSplit, such as $\texttt{n}_2$ in Figure 1(c), splits its query workload $Q_{\texttt{n}}$ into a set of disjoint query subsets $\mathcal{Q} = \{Q_1, \ldots, Q_m\}$, ensuring that queries within the same subset share similar access patterns, while queries across different subsets exhibit distinct access behaviors. For each query subset Q_i, a corresponding child node is created as $\texttt{n.child}_i = (A_{\texttt{n}}, T_{\texttt{n}}, Q_i, O_i)$. Note that a QSplit does not directly compute a joint probability distribution $P_{T_{\texttt{n}}}(A_{\texttt{n}})$ but instead functions as a *query router*. Specifically, when estimating the cardinality of a query q, the QSplit identifies the query subset Q_{k^*} from $\mathcal{Q}$ that shares the most *similar access patterns* with q and routes q to the corresponding child node $\texttt{n.child}_{k^*}$ for CardEst. We will discuss the method for determining whether a query subset exhibits the most *similar access patterns* with q later.

Other Three Types of Nodes. (1) Product is computed as $P_{T_{\texttt{n}}}(A_{\texttt{n}}) = \prod_{i=1}^m P_{T_{\texttt{n}}}(A_i)$, with child nodes $\texttt{n.child}_i = (A_i, T_{\texttt{n}}[A_i], Q_{\texttt{n}}[A_i], O_i)$, ensuring statistical independence between columns in different subsets; (2) Sum is computed as $P_{T_{\texttt{n}}}(A_{\texttt{n}}) = \sum_i \frac{|T_i|}{|T_{\texttt{n}}|} \cdot P_{T_i}(A_{\texttt{n}})$, with child nodes $\texttt{n.child}_i = (A_{\texttt{n}}, T_i, Q_{\texttt{n}}, O_i)$; (3) Leaf is computed as $P_{T_{\texttt{n}}}(A_{\texttt{n}})$, representing the 1-D probability distribution.

To support the above QTree structure, five types of nodes represent different operations to split column set, split data table, split query workload and fit 1-D data distributions. We introduce a framework that consists of both offline and online stages in Section IV and Section V. And we extend QTree to support multi-table CardEst in Section VI.

IV. Offline QTree Construction

Given a table T with column set A and a workload Q, QTree construction generates a QTree that models the joint probability distribution $P_T(A)$ conditioned on Q. To achieve this, the construction process recursively decomposes the joint probability distribution into local probability distributions in a top-down manner. Specifically, during the construction of each node, QTree attempts different node types in the following order: Leaf, Product, QProduct, QSplit, and Sum.

During the recursive process, if $|A| = 1$, this indicates that the joint probability distribution has been fully decomposed into a local distribution over the specific column. In this case, the construction process creates a Leaf to model the 1-D probability distribution $P_{T_n}(A)$. When the column set A of a node exhibits statistical dependencies suitable for partitioning, a Product node is constructed. Following prior works [13], [14], [20], we use the *Randomized Dependence Coefficient* (RDC) [21]. If all other types of nodes fail to meet the decomposition criteria, node n defaults to a Sum, which splits T into subsets T_i. For this, following prior works [13], [14], we use the K-Means clustering algorithm, as it partitions the data into clusters, which helps to reduce data correlation within each subset.

In this section, we illustrate QProduct and QSplit operations in QTree offline construction.

A. QProduct in QTree Offline Construction

QProduct partitions columns according to their query access patterns, grouping frequently co-accessed columns to-

gether while separating those that are rarely co-accessed into distinct subsets.

Formalization of `QProduct`. We first formally define query-column access matrix as follows. Given this workload Q, we introduce *query-column access matrix* (or *access matrix* for short) to represent the **access patterns** of queries in Q on columns in T. This matrix captures which queries reference which columns, enabling more optimization opportunities in query-aware CardEst.

Definition 1 (Query-Column Access Matrix)**.** *For each query q_i and each column a_j, we define $\texttt{acc}(q_i, a_j)$ as an indicator function that specifies whether column a_j is accessed by query q_i. Specifically, $\texttt{acc}(q_i, a_j) = 1$ if q_i accesses a_j, and $\texttt{acc}(q_i, a_j) = 0$ otherwise. Then, we define the* access matrix *for a workload Q and a column set A, denoted as $\textbf{\textit{ACC}}(Q, A)$, as a binary matrix where each entry $\texttt{acc}(q_i, a_j)$ indicates whether query q_i accesses column a_j.*

In particular, we use $\textbf{acc}(q_i)$ to denote the i-th row of access matrix, representing the access pattern of query q_i across all columns, and $\textbf{acc}(a_j)$ to denote the j-th column of access matrix, capturing how column a_j is accessed by queries.

We then define *access affinity* using the query-column access matrix as follows.

Definition 2 (Access Affinity)**.** *The* Access Affinity *between columns a_i and a_j with respect to query workload Q, denoted by $\texttt{AFF}(a_i, a_j|Q)$, is defined as how frequently both columns are referenced together by queries in Q,* i.e.,

$$\texttt{AFF}(a_i, a_j|Q) = \textbf{acc}(a_i) \cdot \textbf{acc}(a_j). \tag{1}$$

We define the `QProduct` operation using access affinity.

Definition 3 (`QProduct`)**.** *Given a workload Q and a column set $A = \{a_1, \ldots, a_{|A|}\}$, `QProduct` partitions A into a set of disjoint column subsets $\mathcal{A} = \{A_1, \ldots, A_m\}$, minimizing the* inter-partition affinity *(*IPA*) of partitioning $\texttt{IPA}(\mathcal{A}|Q)$, where*

$$\texttt{IPA}(\mathcal{A}|Q) = \sum_{1 \le k < l \le m} \sum_{a_i \in A_k} \sum_{a_j \in A_l} \texttt{AFF}(a_i, a_j|Q). \tag{2}$$

For example, consider the workload Q_1 in Figure 1(b). The inter-partition affinity for the column partitioning $\{\{a_1, a_2\}, \{a_3, a_4\}\}$ is 2, whereas for the partitioning $\{\{a_1, a_3\}, \{a_2, a_4\}\}$, it is 6. Based on these results, `QProduct` selects the partition $\{\{a_1, a_2\}, \{a_3, a_4\}\}$ for workload Q_1 to minimize inter-partition affinity.

Algorithm Design for `QProduct`. We first analyze the complexity of the `QProduct` construction problem.

Lemma 1. *The problem of `QProduct` construction is equivalent to the minimum k-cut problem.*

The minimum k-cut problem, even for fixed k, is computationally expensive to solve. We abstract each column to a vertice. For example, the minimum 2-cut can be solved in $O(|A|^3)$ time using algorithms such as Stoer-Wagner [22]. For $k = 3$, the time complexity is $O(|A|^3\tau(|A|))$ [23], where $\tau(|A|)$ represents the cost of computing the objective function, which is $O(|A|^2)$ in our `QProduct` construction problem, yielding an overall complexity of $O(|A|^5)$. Similarly, a recent algorithm for $k = 4$ achieves a complexity of $O(|A|^6\tau(|A|))$ (or $O(|A|^8)$). While the problem is polynomial-time solvable for fixed $k \ge 5$, the complexity increases dramatically (*e.g.,* $O(|A|^{16})$) for a minimum 5-cut algorithm as suggested by [24], rendering such algorithms impractical for real-world use. Even a common approximation algorithm based on Gomory-Hu Tree is too slow ($O(|A|^5)$) for construction with many `QProduct` tries.

Given the computational expense of partitioning the column set to minimize IPA, we design an algorithm **`PartitionByAFF`** that achieves effective and efficient ($O(|A|^2)$) results. The algorithm first constructs a graph $G = (A, E)$, where vertices A represent columns, and an edge $e_{ij} \in E$ connects a_i and a_j if their affinity $\texttt{AFF}(a_i, a_j|Q)$ exceeds a threshold τ. Next, the algorithm identifies the connected components of G, with the vertices in each connected component $G_i \subseteq G$ forming a column partition A_i.

B. `QSplit` in `QTree` Offline Construction

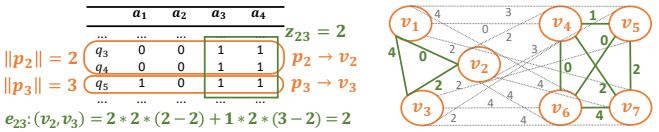
When considering the entire workload Q, `QProduct` may struggle to derive a meaningful column partitioning with a sufficiently low IPA score due to the presence of queries exhibiting diverse access patterns. The following example illustrates this challenge.

Example 2. *Given the workload $Q = \{q_1, q_2, \ldots, q_{10}\}$, as shown in Figure 1(b), an optimal column partitioning for `QProduct` with $k = 2$ results in $\{\{a_1, a_2, a_3\}, \{a_4\}\}$, yielding a minimum IPA score of 6, which remains relatively high. The primary reason for this is that different subsets of queries within Q exhibit different access patterns, leading to conflicting preferences for column partitioning. Specifically, queries in $Q_1 \subset Q$ favor the column partitioning $\{\{a_1, a_2\}, \{a_3, a_4\}\}$, while those in $Q_2 \subset Q$ prefer $\{\{a_1, a_3\}, \{a_2, a_4\}\}$. These conflicting preferences arise due to the distinct access patterns exhibited by queries in different subsets.*

Formalization of `QSplit`. To address this challenge, we introduce the `QSplit` operation, which partitions the workload Q into n subsets, $\mathcal{Q} = \{Q_1, \ldots, Q_n\}$, ensuring that each subset exhibits more consistent access patterns, thus enabling `QProduct` to derive more meaningful column partitions.

Definition 4 (`QSplit`)**.** *Given a query workload Q and a column set $A = \{a_1, a_2, \ldots, a_{|A|}\}$, `QSplit` aims to partition Q into a set of disjoint query subsets $\mathcal{Q} = \{Q_1, Q_2, \ldots, Q_n\}$, minimizing the objective $\sum_{k=1}^{n} \texttt{IPA}(\mathcal{A}_k^*|Q_k)$, where $\mathcal{A}_k^*$ is the* optimal *column partition given query subset Q_k.*

For example, by splitting Q in Figure 1(b) into two subsets ($n = 2$), an effective partitioning results in $Q_1 = \{q_1, \ldots, q_5\}$ and $Q_2 = \{q_6, \ldots, q_{10}\}$. Given these subsets, the optimal `QProduct` column partitioning for Q_1 is $\mathcal{A}_1^* = \{\{a_1, a_2\}, \{a_3, a_4\}\}$ with $\texttt{IPA}(\mathcal{A}_1^*|Q_1) = 2$ and that for Q_2 is $\mathcal{A}_2^* = \{\{a_1, a_3\}, \{a_2, a_4\}\}$ with $\texttt{IPA}(\mathcal{A}_2^*|Q_2) = 2$.



(a) Upper bound design. (b) Our greedy algorithm.

Fig. 2. An example of QSplit construction.

Algorithm Design for QSplit. The QSplit construction problem is highly challenging because it requires simultaneously minimizing $\texttt{IPA}(\mathcal{A}_k^*|Q_k)$ for each partitioned subset Q_k. Thus, instead of directly optimizing $\texttt{IPA}(\mathcal{A}_k^*|Q_k)$ for each Q_k, we focus on minimizing its upper bound, denoted as $\overline{\texttt{IPA}}(\mathcal{A}_k|Q_k)$, which allows us to design a more tractable algorithm while still ensuring effective workload partitioning. Formally, the objective is to partition Q into $\mathcal{Q} = \{Q_1, Q_2, \ldots, Q_n\}$ while minimizing the upper bound of inter-partition affinity, *i.e.,* $\sum_{k=1}^{n} \overline{\texttt{IPA}}(\mathcal{A}_k|Q_k)$.

Next, we design a mechanism to construct the upper bound $\overline{\texttt{IPA}}(\mathcal{A}_k|Q_k)$ whose optimization remains NP-hard. Finally, we propose an efficient heuristic algorithm that achieves effective results.

Upper-bound design. Given a specific query set Q, we construct an upper bound $\overline{\texttt{IPA}}(\mathcal{A}|Q)$ for the optimal column partitioning result $\texttt{IPA}(\mathcal{A}^*|Q)$ by analyzing the access patterns of queries in Q. Since different queries (*e.g.,* q_1 and q_2 in Figure 1(b)) may share the same access pattern (e.g., $(1,1,0,0)$), we introduce p_i to denote the i-th distinct access pattern in Q and let n_i represent the number of queries in Q that follow pattern p_i. For instance, in Figure 1(b), the first access pattern in Q is $p_1 = (1,1,0,0)$, which corresponds to two queries, *i.e.,* $n_1 = 2$. Based on these notations, we construct the following upper bound for $\texttt{IPA}(\mathcal{A}^*|Q)$.

$\overline{\texttt{IPA}}(\mathcal{A}|Q)$. Given a Q consisting of m distinct query patterns $\{p_1, \ldots, p_m\}$, let $\|p_i\|$ denote the L_1 norm of pattern p_i, n_i represent the number of queries corresponding to pattern p_i, and z_{ij} denote the dot product of two patterns p_i and p_j. An upper bound for $\texttt{IPA}(\mathcal{A}^*|Q)$:

$$\overline{\texttt{IPA}}(\mathcal{A}|Q) = \sum_{i<j} n_i \cdot z_{ij} \cdot (\|p_i\| - z_{ij}) + n_j \cdot z_{ij} \cdot (\|p_j\| - z_{ij}).$$

Consider a Q with two query patterns (e.g., p_2 and p_3 in Figure 2(a)). To compute $\overline{\texttt{IPA}}(\mathcal{A}|Q)$, we calculate the dot product of p_2 and p_3: $z_{23} = p_2 \cdot p_3 = 2$, as well as the norms $\|p_2\| = 2$ and $\|p_3\| = 3$. Given that $n_2 = 2$ and $n_3 = 1$, the contribution of p_2 and p_3 to the overall upper bound is $n_2 \cdot z_{23} \cdot (\|p_2\| - z_{23}) + n_3 \cdot z_{23} \cdot (\|p_3\| - z_{23}) = 2$. As shown in Figure 2(a), the upper bound corresponds to a specific column partitioning strategy, *i.e.,* grouping columns that are co-accessed by multiple query patterns (*e.g.,* a_3 and a_4) while placing the remaining columns (*e.g.,* a_1 and a_2) in a separate group.

Lemma 2. *For QSplit, $\overline{\texttt{IPA}}(\mathcal{A}|Q)$ provides an upper estimate of the optimal result $\texttt{IPA}(\mathcal{A}^*|Q)$, given query set Q.*

Hardness of upper-bound optimization. Next, we show that even optimizing $\overline{\texttt{IPA}}(\mathcal{A}|Q)$ is theoretically intractable.

Lemma 3. *For QSplit, the problem of partitioning Q into $\mathcal{Q} = \{Q_1, Q_2, \ldots, Q_n\}$ to minimize the upper bound of inter-partition affinity,* i.e., $\sum_{k=1}^{n} \overline{\texttt{IPA}}(\mathcal{A}_k|Q_k)$, *is reduced from the NP-hard Max-Cut problem.*

Due to space constraints, the proofs of all lemmas are provided in our technical report [25].

A greedy algorithm for QSplit. Given the time complexity of solving our problem, we design a practical and efficient heuristic algorithm. The algorithm first constructs a graph $G = (V, E)$, where each vertex $v_i \in V$ represents a query pattern p_i, and an edge $e_{ij} \in E$ connects two patterns p_i and p_j with a weight of $n_i \cdot z_{ij} \cdot (\|p_i\| - z_{ij}) + n_j \cdot z_{ij} \cdot (\|p_j\| - z_{ij})$. Next, based on the constructed graph G, the algorithm aims to partition it into n sub-graphs, denoted as $\{G_1, G_2, \ldots, G_n\}$, such that the total weight of the edges *across* the sub-graphs is maximized. To achieve this, it first sorts all vertices in G based on their weighted degree in descending order and initializes n empty sub-graphs. Then, the algorithm iteratively processes each vertex and assigns it to an *appropriate* sub-graph. Specifically, in the i-th iteration, the algorithm considers vertex v_i and evaluates its potential assignment to each sub-graph G_j by computing the total edge weight of G'_j after incorporating v_i. The vertex v_i is then assigned to the sub-graph that results in the minimum weight summation. Figure 2(b) illustrates an example of our greedy algorithm partitioning the query set Q from Figure 1(b) into two subsets (*i.e.,* $n = 2$).

V. Online QTree Computation

A. CardEst Inference with QTree

When a new query q arrives, the Online CardEst Inference process using QTree operates recursively by traversing the QTree tree from the root to the leaf nodes, as shown in Figure 1(c). Specifically, during traversal, if a QSplit is visited, the process routes query q to the child node corresponding to the most relevant query subset. On the other hand, if a QProduct, Product, or Sum is visited, the process computes the joint probability according to the joint probability definition $P_{T_n}(A_n)$ in Section III. Next, we explore two key implementation details of the CardEst Inference process: (1) query routing in QSplit nodes, and (2) sub-tree pruning in QProduct, Product, Sum.

Query Routing of QSplit. The objective of query routing in a QSplit node (say $\texttt{n}_2$ in Figure 1(c)) is to measure the degree to which a query set Q shares *similar access patterns* with a given query q, which guides the QSplit node in routing q to the appropriate child node (*e.g.,* node $\texttt{n}_5$). To this end, we formally introduce a matching score $\texttt{S}(Q, q)$ between a query set Q and a query q:

$$\texttt{S}(Q, q) = \frac{1}{|Q|} \sum_{(a_i, a_j), i<j} \texttt{AFF}(a_i, a_j|Q) \cdot \texttt{AFF}(a_i, a_j|\{q\}) \quad (3)$$

The intuition behind the matching score $\texttt{S}(Q, q)$ is to measure how closely the access patterns of queries in Q align

with the access pattern of q. For example, consider the query q executed in Figure 1(c). The access pattern for q is:

$$\texttt{AFF}(\cdot,\cdot|\{q\}) = [[1,0,1,0],[0,0,0,0],[1,0,1,0],[0,0,0,0]].$$

Considering the workload partitioning in Figure 1(b), we have $\texttt{AFF}(\cdot,\cdot|Q_1) = [[3,2,1,1],[2,2,0,0],[1,0,3,3],[1,0,3,3]]$ and $\texttt{AFF}(\cdot,\cdot|Q_2) = [[2,1,2,0],[1,3,1,2],[2,1,3,0],[0,2,0,2]]$ for Q_1 and Q_2, respectively. So we compute $\texttt{S}(Q_1,q) = \frac{1}{5}$ and $\texttt{S}(Q_2,q) = \frac{2}{5}$. Therefore, when the node $\texttt{n}_2$ is visited, the algorithm routes query q to its child node $\texttt{n}_5$, as Q_2 shares more similar access patterns with q than Q_1.

Pruning Rules. We employ pruning rules that leverage query q and pre-computed metadata to exclude irrelevant child nodes of a given node n. (1) For `QProduct` and `Product` nodes, let A_q denote the set of columns constrained by the query q. For a child node $\texttt{n.child}_k$ of n, corresponding to the column subset A_k, pruning occurs if $A_k \cap A_q = \emptyset$. (2) For `Sum` nodes, it decides which child nodes contributes to $n.P_T(q)$ so that participate the computation *i.e.,* set of visited child nodes $\mathcal{C}$. Consider a child node $\texttt{n.child}_k$ of n, corresponding to the table subset T_k. In offline construction, we pre-compute and store the range of values for each column a_i in T_k. During online CardEst, if the value ranges of T_k for any column do not overlap with the constraints specified by query q, the child node $\texttt{n.child}_k$ can be safely pruned, as it does not contribute to the result.

B. `QTree` Model Update

Data updates (ΔT, which include new tuples) and query workload shifts (ΔQ, which include new queries) impact the accuracy and inference efficiency of the original `QTree` model. The high-level idea of the update method is to traverse `QTree` in a top-down manner. Each time a node n is visited during the traversal, two steps are performed to update the subtree rooted at n. First, the method examines whether n, originally constructed using $\texttt{n}.T$ and $\texttt{n}.Q$, still fits $\texttt{n}.T \cup \Delta T$ or $\texttt{n}.Q \cup \Delta Q$. Second, if n no longer fits, the subtree rooted at n is reconstructed by calling the `QTree` construction method (see Section IV), which generates a new subtree rooted at $\texttt{n}'$. Otherwise, each child $\texttt{n}.child_i$ is recursively updated. Note that the check and reconstruction steps are unnecessary for `Leaf` as histograms can be incrementally updated in $O(|\Delta T|)$.

The key challenge in the above update method is efficiently examining whether a node n still fits $\texttt{n}.T \cup \Delta T$ or $\texttt{n}.Q \cup \Delta Q$, as the corresponding data table $\texttt{n}.T$ and query workload $\texttt{n}.Q$ are not materialized at node n. To address this challenge, we maintain lightweight data structures in different types of nodes and design a mechanism to check whether node n is still up-to-date with respect to the data and query workload:

(1) If n is a `QProduct` node, we examine whether the column partitioning still holds, *i.e.,* whether columns in different partitions remain infrequently co-accessed with respect to the updated workload $Q \cup \Delta Q$. To do this, we first compute $\texttt{AFF}(a_i, a_j|\Delta Q)$, where $a_i, a_j \in \texttt{n}.A$, and then check whether there exist a_i, a_j from different child nodes, say a_i from $\texttt{n}.child_k$ and a_j from $\texttt{n}.child_l$ such that $\frac{|\texttt{n}.Q|}{|\texttt{n}.Q|+|\Delta Q|}\texttt{AFF}(a_i,a_j|\texttt{n}.Q) + \frac{|\Delta Q|}{|\texttt{n}.Q|+|\Delta Q|}\texttt{AFF}(a_i,a_j|\Delta Q)$ is larger than a pre-defined threshold. If any such pair a_i, a_j is found, the subtree rooted at n needs reconstructing for more accurate CardEst. If the `AFF` between columns within a child node becomes less significant due to ΔQ, we may reconstruct the subtree rooted at the child node to further partition the columns that are infrequently co-accessed now.

(2) If n is a `QSplit`, we examine whether the query routing strategy still holds for the updated workload $Q \cup \Delta Q$. To do this, for each workload partition Q_k corresponding to the child node $\texttt{n}.child_k$, we maintain the average of the matching scores of the queries in Q routed to Q_k, i.e., $\sum_{q \in Q_k} \texttt{S}(Q_k, q)/|Q_k|$. Then, for each query q' in ΔQ, we assign q' to the child node with the maximum matching score (see Section V-A) and update the average matching score accordingly. If the average matching score of any workload partition becomes less significant, *e.g.,* less than a predefined threshold, we reconstruct the subtree rooted at n, as the workload partition no longer reflects the access patterns of $Q \cup \Delta Q$.

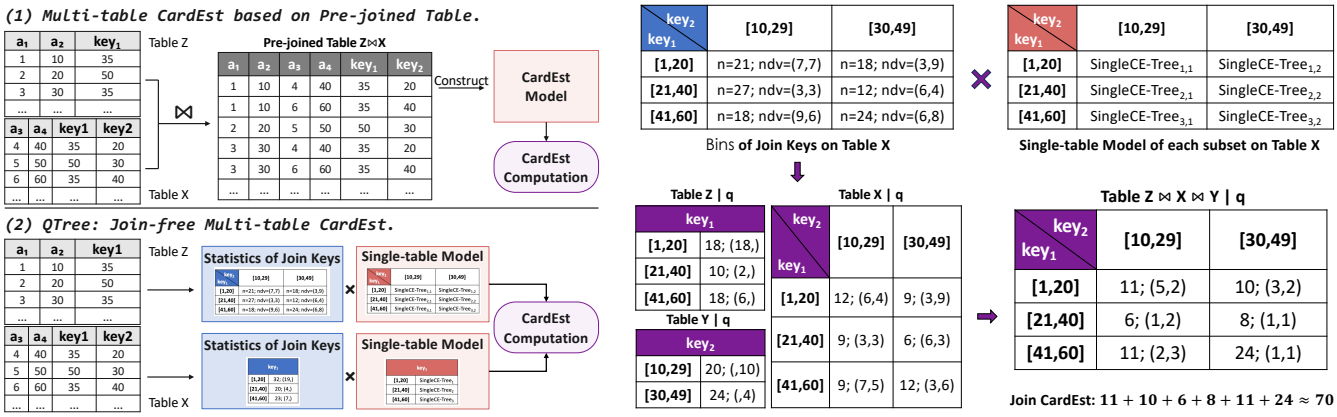
For other types of nodes. If n is a `Product` node, the update examination strategy is similar to that of the `QProduct` case, except that we consider the column correlations $\texttt{RDC}(a_i, a_j|\texttt{n}.T)$ for any column pair (a_i, a_j), instead of the access affinity $\texttt{AFF}(a_i, a_j|\texttt{n}.Q)$. If n is a `Sum` node, we maintain the centroid for each tuple subset T_i from $\mathcal{T} = \{T_1, T_2, \ldots, T_m\}$ and the average distance between each tuple and the centroid of its assigned subset. Then, for each new tuple t in ΔT, we assign t to the tuple subset with the minimum distance to the centroid of that subset and update the average distance accordingly. If the average distance becomes significant, we update the subtree rooted at n, as the `Sum` may no longer hold for $\texttt{n}.T \cup \Delta T$.

In this way, the `QTree` update method minimizes unnecessary cost on model updates while ensuring accuracy in response to data updates and query workload shifts.

VI. Multi-Table CardEst by QTree

We introduce a *join-free* multi-table CardEst method built upon single-table `QTree` models and single-table statistics of join keys. As Figure 3(a) shows, many learned CardEst methods [11], [13], [14] construct models over pre-joined tables for multi-table CardEst, scaling poorly as the number and size of tables increase, while `QTree` collects statistics and constructs models for each table separately, enabling efficient online multi-table CardEst computations.

For clarity, this section firstly considers a query q that performs a star join on two groups of join keys: key_1, key_2 among three tables X, Y and Z. In query q, tables Y and Z are inner joined to the *center table* X on the condition $X.key_1 = Z.key_1 \wedge X.key_2 = Y.key_2$, denoted as $q(Z \bowtie X \bowtie Y)$. We assume that join keys in the same group share the same value domain *i.e.,* $X.key_1$ and $Z.key_1$ share domain D_1; $X.key_2$ and $Y.key_2$ share domain D_2. Besides, query q has base table filter predicates $q(X)$, $q(Y)$ and $q(Z)$. Thus, the CardEst of star join queries (say q) can be formalized as estimating $|q(Z \bowtie$



(a) QTree: Join-free Multi-Table CardEst. (b) An Example: CardEst of Star Join on 2 Groups of Join Keys using QTree

Fig. 3. Illustration of our proposed QTree multi-table CardEst method.

$X \bowtie Y)| = |X||Y||Z| \sum_{v_1 \in D_1} \sum_{v_2 \in D_2} P(X.key_1 = v_1 \wedge X.key_2 = v_2 \wedge q(X)) \cdot P(Z.key_1 = v_1 \wedge q(Z)) \cdot P(Y.key_2 = v_2 \wedge q(Y))$ where $P(X.key_1 = v_1 \wedge X.key_2 = v_2 \wedge q(X))$ denotes the probability that the join keys $X.key_1$ and $X.key_2$ equal v_1 and v_2 respectively in the result table of the base filter predicates $q(X)$. It is similar for $P(Z.key_1 = v_1 \wedge q(Z))$ (or $P(Y.key_2 = v_2 \wedge q(Y))$).

As directly estimating $|q(Z \bowtie X \bowtie Y)|$ is computationally expensive, QTree supports multi-table CardEst by *binning join keys*: dividing the domains of join keys into equal-width **bins** organized into a k-D grid for a table involving k groups of join keys. Specifically, on table X, D_1 and D_2 of the join keys key_1 and key_2 are divided into a set of **bins**: $\mathbf{Bins}_X = \{\mathtt{Bin}_{1,1}, \ldots, \mathtt{Bin}_{1,n_2}, \mathtt{Bin}_{2,1}, \ldots, \mathtt{Bin}_{n_1,1}, \ldots, \mathtt{Bin}_{n_1,n_2}\}$. Similarly, we have $\mathbf{Bins}_Z = \{\mathtt{Bin}_1, \ldots, \mathtt{Bin}_{n_1}\}$ and $\mathbf{Bins}_Y = \{\mathtt{Bin}_1, \ldots, \mathtt{Bin}_{n_2}\}$. To estimate $|q(Z \bowtie X \bowtie Y)|$ using the bins, we have:

$$|q(Z \bowtie X \bowtie Y)| = |X||Y||Z| \sum_{\mathtt{Bin}_{i_1,i_2} \in \mathbf{Bins}_X} \sum_{v_1, v_2 \in D_{\mathtt{Bin}_{i_1,i_2}}} P(Z.key_1 = v_1 \wedge q(Z)) \cdot P(Y.key_2 = v_2 \wedge q(Y)) \cdot P(X.key_1 = v_1 \wedge X.key_2 = v_2 \wedge q(X)), \tag{4}$$

where $v_1 \in D_{\mathtt{Bin}_{i_1} \in \mathbf{Bins}_Z}$ is satisfied for $P(Z.key_1 = v_1 \wedge q(Z))$ and $v_2 \in D_{\mathtt{Bin}_{i_2} \in \mathbf{Bins}_Y}$ is satisfied for $P(Y.key_2 = v_2 \wedge q(Y))$. Then, the task is to estimate $P(X.key_1 = v_1 \wedge X.key_2 = v_2 \wedge q(X)) \cdot P(Z.key_1 = v_1 \wedge q(Z)) \cdot P(Y.key_2 = v_2 \wedge q(Y))$ in Equation (4). To achieve this, we maintain statistics representing each bin of join keys. Formally, we define a bin, a cell of a k-D grid on a table involving k groups of join keys (say $X.key_1, X.key_2$), as $B = (\mathtt{range}, \mathtt{num}, \mathtt{ndv})$, where range is a k-tuple representing the identifier and domain of the bin B for matching bins across tables, num is the number of tuples in B, ndv is a k-tuple representing the number of distinct values of each join key in B. Besides, we have a global width determining the binning granularity, which also limits the maximum distinct values of each join key in a bin. Then, we organize bins: $\mathcal{B}_X = \{B_{1,1}, \ldots, B_{n_1,n_2}\}$ on table X, $\mathcal{B}_Z = \{B_1, \ldots, B_{n_1}\}$ on table Z and $\mathcal{B}_Y = \{B_1, \ldots, B_{n_2}\}$ on table Y.

Example 3. *The 2-D grid in the upper right corner of Figure 3(b) shows an example of constructing the aforementioned bins on table X where the global* $\mathtt{width} = 20$*: the bin $B_{3,1} \in \mathcal{B}_X$, corresponding to* range *of* $key_1 \in [41, 60], key_2 \in [10, 29]$*, has* $\mathtt{num} = 18$*,* $\mathtt{ndv} = (9, 6)$*. Similarly, we have the bin $B_3 \in \mathcal{B}_Z$ and the bin $B_1 \in \mathcal{B}_Y$. In the inference, we change the statistics by query filtering predicates to get new bins $B_{3,1} \in \mathcal{B}_{X|q(X)}, B_1 \in \mathcal{B}_{Y|q(Y)}, B_3 \in \mathcal{B}_{Z|q(Z)}$ as shown in the lower left corner of Figure 3(b). Then, we estimate the statistics after joining: $B_{3,1} \in \mathcal{B}_{Z \bowtie X \bowtie Y|q}$ (shown in the lower right corner of Figure 3(b)) of which the $B_{3,1}.\mathtt{num} = 11$ is the estimate of $|X||Y||Z| \sum_{v_1, v_2 \in D_{Bin_{3,1}} \in \mathcal{B}_X} \cdot$ in Equation* (4)*. Finally, summing* num *of all bins after filtering and joining yields a CardEst of query q:* 70*.*

In this section, we address two challenges in the above process. Firstly, while computing statistics for a given bin B on a table is straightforward, the challenge lies in estimating $\sum_{v_1, v_2 \in B_{3,1} \in \mathcal{B}_X} P(X.key_1 = v_1 \wedge X.key_2 = v_2 \wedge q(X)) \cdot P(Z.key_1 = v_1 \wedge q(Z)) \cdot P(Y.key_2 = v_2 \wedge q(Y))$ because some tuples in the matching bins across tables are not in the joined table. And the base table filtering predicates (e.g. $q(X)$) also change the statistics in each bin. Secondly, the aforementioned equations do not directly apply in join forms beyond a simple star join (e.g. $T.key_2 = S.key_2 \wedge S.key_1 = X.key_1 \wedge X.key_2 = Y.key_2 \wedge X.key_1 = Z.key_1$) since some tables are not joined directly (e.g. table T and table X) in a query. In this case, there are no value variables of join keys (say $v_1, v_2, \ldots$) that are shared by all tables. The following two subsections present our approach to addressing these challenges.

A. Joining Estimation with Base Table Filtering

For base table filtering, we partition each table T into subsets corresponding to join keys binning and construct a single-table model on each table subset, like $\mathcal{X} = \{X_{1,1}, \ldots, X_{3,2}\}$ shown in the upper right corner of Figure 3(b). And we also extract a column set A' from the column set A of each table, containing the columns used for table filtering. So we can estimate the base table filtering selectivity (probability) of each subset T_i of each table to change the statistics of bins. Then, we propose how to estimate the statistics of bins after joining.

Base Table Filtering. A QTree model on (A', T_i) computes the single-table probability $P_{T_i}(A')$ by predicates $q(T)$. The statistics of a bin B_i are then changed by $P_{T_i}(A')$ as follows.

(1) range: It is fixed for matching bins *i.e.,* not changed.

(2) num: Naturally, $B_i.\texttt{num}$ is changed to $B_i.\texttt{num} \cdot P_{T_i}(A')$.

(3) ndv: The change of $B_i.\texttt{ndv}$ after filtering is based on a natural assumption that values of join keys are selected randomly according to single-table probability. For each join key key_j, the average frequency of each value is $B_i.\texttt{num}/B_i.\texttt{ndv}_j$. Since each tuple is filtered with probability $1 - P_{T_i}(A')$, the probability that a join-key value remains after filtering is $1 - (1 - P_{T_i}(A'))^{\texttt{num}/\texttt{ndv}_j}$. Therefore, we have new $B_i.\texttt{ndv}_j$ after filtering as $B_i.\texttt{ndv}_j \cdot \left[1 - (1 - P_{T_i}(A'))^{B_i.\texttt{num}/B_i.\texttt{ndv}_j}\right]$.

Joining Estimation. After base table filtering, given a bin B_i of the center table T which covers k groups of join keys, we have a set of matching bins $\mathcal{B}^j = \{B_1, \ldots, B_{n_j}\}$ on other tables which are joined to table T on the join key key_j where $\forall B \in \mathcal{B}^j, B.\texttt{range} = B_i.\texttt{range}_j$. As Example 3 shows, for the bin $B_{3,1}$ of center table X which covers 2 groups of join keys, we have $\mathcal{B}^1 = \{B_3\}$ on table Z (*i.e.,* $B_3 \in \mathcal{B}_Z$) and $\mathcal{B}^2 = \{B_1\}$ on table Y (*i.e.,* $B_1 \in \mathcal{B}_Y$). Based on this, we propose the estimates of statistics after joining.

(1) ndv: For each key_j, only values which appear in all bins of the $\mathcal{B}^j$ are in the joined table. And since there is a global width deciding the maximum distinct values in a bin, we convert the problem of estimating $\texttt{ndv}_j$ after joining to estimating size of the intersection containing $B_1.\texttt{ndv}, \ldots, B_{n_j}.\texttt{ndv}$ elements. Thus, the $\texttt{ndv}_j$ after joining is estimated as $\texttt{width} \cdot \frac{B_i.\texttt{ndv}_j}{\texttt{width}} \cdot \prod_{B \in \mathcal{B}^j} \frac{B.\texttt{ndv}}{\texttt{width}}$ which is simplified as $\frac{B_i.\texttt{ndv}_j}{\texttt{width}^{n_j}} \prod_{B \in \mathcal{B}^j} B.\texttt{ndv}$.

(2) num: Assuming all values of join keys appear in the joined table *i.e.,* $B_i.\texttt{ndv}$ is not changed after joining, each tuple in the center table T repeats the product of average frequency of each value in a table joined to T. But this calculation causes overestimation since the $B_i.\texttt{ndv}$ can be reduced a lot according to the formula of $B_i.\texttt{ndv}$ above. Thus, we multiply the formula by a downscale factor $scale$ to take account of the effect of $B_i.\texttt{ndv}$ change on the $B_i.\texttt{num}$ after joining: $scale \cdot B_i.\texttt{num} \prod_{1 \le j \le k} \prod_{B \in \mathcal{B}^j} \frac{B.\texttt{num}}{B.\texttt{ndv}}$. And $scale$ can be estimated by comparing the $B_i.\texttt{ndv}$ after joining (denoted as $B_i.\texttt{ndv}'$) to the original $B_i.\texttt{ndv}$: $scale = \prod_{1 \le j \le k} (\frac{B_i.\texttt{ndv}'_j}{B_i.\texttt{ndv}_j})^{\frac{1}{j}}$ where the ratios $\frac{B_i.\texttt{ndv}'_j}{B_i.\texttt{ndv}_j}$ are sorted in ascending order. The exponent $1/j$ accounts for the correlation among different groups of join keys.

For CardEst of the star join to the center table T, the result is $\sum_{B_i \in \mathcal{B}_T} B_i.\texttt{num}$ after filtering and joining.

B. CardEst for Complex Join Forms

We propose a *CardEst Graph* which decomposes complex join forms into a series of star joins for CardEst computation.

Example 4. *Given a query with join predicates: $T.key_2 = S.key_2 \wedge S.key_1 = X.key_1 \wedge X.key_2 = Y.key_2 \wedge X.key_1 = Z.key_1$, the CardEst cannot be computed by star join formula above simply since the table T and table X are not joined directly. The CardEst graph (Y-X, S-T with edge key_2 and Z-X, X-S with edge key_1) decomposes this query into triple star joins. First, table Y and table Z are joined to the center table X. Second, table $Z \bowtie X \bowtie Y$ is joined to the center table S. Third, table $Z \bowtie X \bowtie Y \bowtie S$ is joined to the center table T.*

CardEst Graph. Given a query, we construct a CardEst graph by its join predicates, of which each node v represents a table T and each edge e represents the join key between two tables. Based on this, CardEst is performed by traversing the graph in arbitrary order since the equality conditions are transitive.

When traversing the CardEst graph like Example 4, all edges of the graph become *directed* where a node Y pointing to a node X means joining table Y to X, thereby all nodes have zero or more in-edge (as the center table of a star join), and only one out-edge (be joined to the center table of another star join) except the final node for CardEst result (e.g. node T). Once the traversal order of the CardEst graph is fixed, each node v is represented by $(center, parent, child)$, where $v.center$ denotes its associated table; $v.parent = (v.parent.v, v.parent.e)$ specifies the node to which v points and the corresponding join key; and $v.child = \{(v.child_i.v, v.child_i.e)\}_{i=1}^{m}$ specifies the nodes pointing to v and their corresponding join keys.

The join form is decomposed into several star joins, each processed in two steps: star join to the center table $v.center$; bins projection for another star join whose center table is $v.parent.v.center$ (to a 1-D grid only covering $v.parent.e$).

Bi-Pruning of Bins Matching. When processing star joins, some bins of $v.center$ may not be matched by $v.child$, making the computation of their single-table probabilities unnecessary. So we introduce *Bi-Pruning*. Before visiting $v.child$, we prune bins of $v.center$ that are not matched by all $v.child$. After finishing calculations on $v.child$, we again prune bins of $v.center$ whose one of the matched bins of $v.child$ yields zero single-table probability.

Bins Projection is essentially merging bins that share the same $\texttt{range}_{v.parent.e}$. Given such bins $B_1, \ldots, B_n$, the merged bin B is computed as follows: (1) $B.\texttt{range} = B_i.\texttt{range}_{v.parent.e} | \forall B_i$. (2) $B.\texttt{num} = \sum_{B_i} B_i.\texttt{num}$. (3) $B.\texttt{ndv}$ is 1-tuple, recording the number of distinct values of B on the join key $v.parent.e$. We have $B.\texttt{ndv} = \texttt{width} \cdot \left[1 - \prod_{B_j} \left(1 - \frac{B_j.\texttt{ndv}_{v.parent.e}}{\texttt{width}}\right)\right]$.

VII. Experiments

A. Experimental Setup

Datasets. We evaluate our proposed QTree on single-table and multi-table CardEst datasets. Table II provides the statistics of the datasets.

Single-table Datasets. We use three datasets, which are commonly used in existing CardEst studies [10], [14].

- *GAS* [26] is a real-world gas sensing dataset, and we extract the most informative 8 columns (Time, Humidity,

TABLE II
STATISTICS OF DATASETS.

Dataset	Tables	Tuples	Columns (Numeric)	Domain Size
GAS	1	4M	8 (8)	$10^1 \sim 10^2$
Forest	1	0.6M	54 (10)	$10^2 \sim 10^3$
Power	1	2M	9 (7)	$10^4 \sim 10^5$
IMDB	21	74M	108 (57)	$10^0 \sim 10^7$
STATS	7	1M	43 (43)	$10^0 \sim 10^{10}$

Temperature, Flow_rate, Heater_voltage, R1, R5, and R7), following the existing works [14], [27].

- *Forest* [28] is a real-world forest dataset from the US Forest Service and US Geological Survey.
- *Power* [29] is an electric power consumption dataset consisting of measurements gathered in a house located in Sceaux between December 2006 and November 2010.

Multi-table Datasets. We use two multi-table datasets.

- *IMDB* [30] is a real dataset containing information about movies with a star join form. It is extensively used in prior work for multi-table CardEst evaluation [1], [11], [13], [14], [31]. The columns in *IMDB* typically have large domain sizes, which increases the difficulty of CardEst.
- *STATS* [32] is a real dataset recording user-generated content in Stats Stack Exchange, which has more complex distribution and join forms (star + chain) than *IMDB*.

Query Workloads. We describe how the query workloads used in the experiments are constructed.

Synthetic Workloads for Single-Table CardEst. Since no real query workloads are available for the three single-table datasets, we synthesize workloads as follows.

(1) *Template generation*: Following existing DBMS benchmarks [33]–[35], we first generate SQL templates with different column combinations as query predicates.

(2) *Template selection*: Since data correlation strongly affects CardEst performance according to prior studies [8], [10], [13], [14], [16], we group templates into two groups by accessing highly correlated columns or weakly correlated columns. We then sample templates evenly from both groups.

(3) *Query synthesis*: Given a template, we generate queries by instantiating it with randomly selected predicates. Previous studies [10], [16], [19] show that predicates significantly affect CardEst performance. Thus, following [16], we generate predicates in two steps: selecting a random tuple as the range center; determining the range width using a uniform distribution and an exponential distribution in a 1:1 mixture.

Data Updates and Workload Shifts. We generate a read-write hybrid workload where each SQL statement is randomly designated as either a query or a DML command. First, we create a training workload with queries accessing only weakly correlated columns and range constraints following a uniform distribution. Then, we add at least 20% new data tuples with increased correlation by sampling from a table with sorted columns, and include the tuples as DML commands. Finally, we construct a test workload consisting of (1) queries following the original patterns and distribution and (2) queries accessing highly correlated columns with range constraints following an exponential distribution.

Real-World Workloads for Multi-Table CardEst.

- For the multi-table dataset *IMDB*, we use *JOB-light* [35], a widely used multi-table workload for CardEst. It has 70 queries on *IMDB*, each containing star joins over tables, with multiple range constraints. We use *JOB-light* as the test workload and adopt an extended workload provided by existing methods [8], [36] as the training workload.
- For the multi-table dataset *STATS*, we use *STATS-CEB* [32], containing 146 queries on *STATS*. The queries cover multiple join forms (chain, star and mixture) with varied single-table selectivities. We randomly split *STATS-CEB* into two halves as test and training workloads respectively. Both workloads contain all column patterns and join forms.

Baselines. We compare `QTree` against the following methods.

Sampling [16] is a sampling-based cardinality estimator whose size is tuned to match that of `QTree`.

MSCN [8] is a query-driven CardEst model based on regression models. We use its implementation from [16] and set its hyper-parameters following the configurations in [16].

Naru [10] is a DAR-based model that fits the joint data distribution for CardEst. We use the implementation of Naru and set its hyper-parameters following the configurations in [16].

NeuroCard [11] is a multi-table CardEst extension method of Naru. It trains DAR models on pre-joined tables.

DeepDB [13] is an SPN-based model that fits joint data distribution for CardEst under a local independence assumption. We use the implementation from [16] and follow its hyperparameter settings. We also apply it for multi-table CardEst.

FLAT [14] is a CardEst method based on DeepDB, which introduces factorization and multi-dimensional histograms. We use the implementation provided by the authors of FLAT [27] and tune its hyper-parameters as recommended in the original paper. We also use it as a multi-table CardEst method.

UAE [19] is a hybrid-driven CardEst method based on Naru that additionally incorporates supervised losses from queries. We use the authors' single-table CardEst implementation [36].

FactorJoin [31] is a multi-table CardEst method based on join-keys binning without modeling data distribution on generated outer-join tables. We use the implementation [37] as a multi-table CardEst method.

Evaluation Metrics. We use the following metrics to comprehensively evaluate and compare CardEst methods.

Estimation Accuracy. For each query, we measure estimation accuracy using Q-error, defined as the ratio between the estimation est and the ground truth gt, *i.e.,* $Q\text{-error} = \frac{\max\{est,gt\}}{\min\{est,gt\}}$.

Inference Time. For each query in the test set, we measure the time spent for CardEst as inference time. We then compute the mean inference time among the queries in the test set.

Construction Time and Storage Overhead. We measure the total time spent for constructing or learning the models as the

TABLE III
EVALUATING CARDEST ON SINGLE-TABLE AND MULTI-TABLE DATASETS.

Dataset & Workload		Method	Estimation Errors					Inference Time (ms)	Construction Time (min.)	Model Size (MB)
			50th	90th	95th	99th	Mean			
Single-table	GAS	Sampling	1.06	1.42	1.87	240.88	9.03	1.06	**0.002**	0.217
		MSCN	1.46	3.61	6.45	22.21	2.65	0.386	0.157	4.423
		Naru	1.07	1.26	1.39	1.87	1.12	13.658	149.205	1.923
		DeepDB	1.06	9.32	14.22	48.23	3.94	8.22	3.316	0.580
		FLAT	**1.02**	1.7	2.12	3.12	1.21	1.209	0.810	13.996
		UAE	1.06	**1.21**	**1.30**	**1.71**	**1.10**	2.013	301.390	0.747
		QTree	1.06	1.69	2.12	3.12	1.22	**0.301**	1.446	**0.199**
	Forest	Sampling	1.07	**1.44**	1.89	25.01	2.64	0.88	**0.001**	1.312
		MSCN	1.49	3.53	5.36	11.73	2.25	**0.427**	0.128	1.501
		Naru	1.17	1.79	2.31	7.39	1.50	38.651	80.026	2.216
		DeepDB	1.07	1.57	2.02	4.81	1.88	11.295	2.562	1.64
		FLAT	**1.02**	1.48	**1.80**	3.67	**1.30**	41.69	1.821	972.307
		UAE	1.16	1.82	2.75	12.96	1.6	76.721	188.661	29.147
		QTree	1.05	1.48	1.84	**3.08**	1.81	0.77	2.167	**1.261**
	Power	Sampling	1.04	1.39	3.88	60.96	3.96	1.01	**0.001**	0.634
		MSCN	1.88	8.31	15.58	32.45	4.32	0.421	0.144	4.402
		Naru	1.06	1.35	1.61	3.48	1.19	43.125	175.368	4.327
		DeepDB	1.04	3.42	5.45	198.2	13.05	12.731	3.722	3.711
		FLAT	**1.01**	**1.22**	**1.39**	**2.27**	1.27	2090.254	2.225	10100.46
		UAE	1.04	1.27	1.58	4.15	**1.16**	18.22	1316.239	12.599
		QTree	1.05	1.73	2.43	6.33	1.33	**0.281**	0.782	**0.623**
Multi-table	JOB-light	DeepDB	1.69	2.79	**3.71**	15.53	**2.24**	43.674	58.951	34.428
		FLAT	**1.40**	**2.68**	5.41	22.65	2.46	8.595	51.791	3.391
		NeuroCard	1.65	4.39	6.97	**10.8**	2.51	608.3	60.051	6.93
		FactorJoin	3.98	21.37	36.00	88.91	10.20	3.97	25.22	**1.47**
		QTree	1.50	4.66	6.67	12.27	2.32	**3.961**	**6.06**	135.895
	STATS-CEB	DeepDB	**2.28**	69.44	213.26	22471.59	1103.49	87.457	108.546	162.565
		FLAT	3.69	39.10	127.88	22571.58	1112.70	175.133	262.786	310.271
		NeuroCard	1194.60	$1.2 * 10^7$	$2.4 * 10^8$	$8.0 * 10^8$	$3.9 * 10^7$	822.349	568.699	336.272
		FactorJoin	5.40	84.52	594.59	3144.47	151.41	**13.11**	**1.278**	**6.212**
		QTree	2.93	**12.80**	**25.15**	**49.48**	**6.09**	18.964	3.025	27.579

construction time and the total size of the model or statistics for each method on each dataset as the storage overhead.

End-to-End Query Time. We measure CardEst time and query execution time. Following the benchmark setting in [32], we first compute the CardEst for all subqueries using each method to measure CardEst time. We then inject the estimated cardinalities into PostgreSQL [1] by replacing the optimizer's CardEst with the predicted values and measure the execution time of the query workload. Finally, we compare the end-to-end query time, defined as the sum of CardEst and query execution time, across all methods.

Experimental Settings. All methods are implemented in Python 3.7. Experiments are conducted on an AMD-64 server: Ubuntu 20.04.6 LTS, 2× Intel(R) Xeon(R) Gold 6230 CPU @ 2.10GHz (20C40T), 1TB DDR4 ECC memory, 4×8 TB HDD (RAID5) Storage. We use the same default hyper-parameters of QTree across all datasets: $\tau = 0.01$ (normalized by the size of training workload) which corresponds to the QProduct. For multi-table CardEst experiments, we set $width = 1{,}500{,}000$ for *IMDB* and $width = 60{,}000$ for *STATS* based on the join-key domain sizes.

B. Evaluation on Single-Table CardEst

This section evaluates QTree on the single-table CardEst datasets. Figure 4 presents a Pareto comparison of all methods on the three single-table datasets with respect to estimation accuracy, inference time, and storage overhead. Each method is represented as a point *(mean Q-error, inference time)*, with storage overhead annotated for each point. And detailed results are shown in Table III.

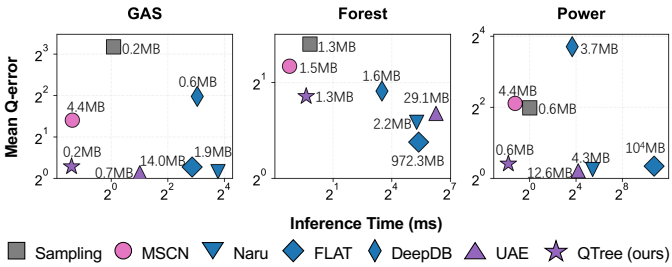
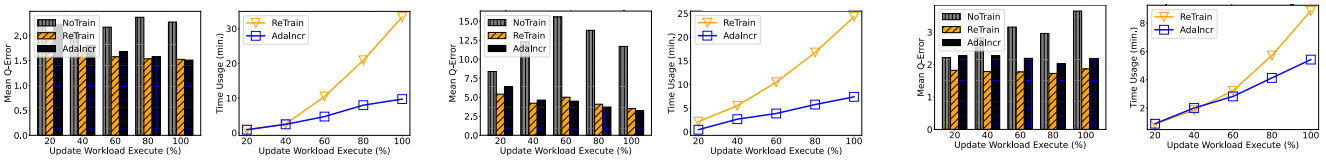


Fig. 4. Pareto trade-offs in Single-table CardEst.

QTree lies on the *Pareto frontier* across the evaluated datasets, balancing estimation accuracy, inference time and storage overhead. For example, on *GAS*, QTree achieves a 1.22 mean Q-error, 0.301 ms inference, 0.199 MB storage overhead, with no baseline improving all three metrics. It matches comparable estimation accuracy of data-driven *FLAT*, *Naru* and hybrid-driven *UAE* at lower inference time and storage overhead, whereas query-driven *MSCN* trades accuracy for faster inference than data-driven methods, the traditional sampling method is both less accurate and slower than QTree, and SPN-based model *DeepDB* offers a weaker accuracy–latency trade-off on highly correlated data.

The offline construction time is negligible for traditional method *Sampling* and query-driven *MSCN*, minutes for tree-structured *DeepDB*, *FLAT* and QTree, and hours for DAR-based *Naru* and *UAE*.



(a) Dataset GAS. (b) Dataset Forest. (c) Dataset Power.

Fig. 5. Accuracy and Time Usage of QTree on Hybrid-Workload containing Data Updates and Workload Shifts.

TABLE IV
MEAN Q-ERRORS IN DIFFERENT JOIN FORMS OF STATS-CEB.

Join Form	NeuroCard	DeepDB	FLAT	FactorJoin	QTree
Star	$5.59 * 10^7$	21.83	19.22	125.15	**3.77**
Chain	$3.22 * 10^4$	85.05	83.79	188.86	**3.68**
Mixture	$7.25 * 10^3$	$4.39 * 10^3$	$4.44 * 10^3$	217.5	**11.96**

TABLE V
EVALUATING END-TO-END PERFORMANCE.

Benchmark	Method	CardEst Time (s)	Query Exec. Time(s)	Total Time(s)	Saved
JOB-light	PostgreSQL	/	4122	4122	/
	DeepDB	30.397	4035	4065.397	1.4%
	FLAT	5.982	4087	4092.982	0.7%
	NeuroCard	423.377	**3544**	3967.377	3.8%
	FactorJoin	2.763	3824	3826.763	7.2%
	QTree	**2.756**	3606	**3608.756**	**12.5%**
STATS-CEB	PostgreSQL	/	11613	11613	/
	DeepDB	227.651	10683	10910.651	6.0%
	FLAT	455.871	10304	10759.871	7.3%
	NeuroCard	2140.574	12799	14939.574	-28.6%
	FactorJoin	**34.125**	10866	10900.125	6.1%
	QTree	49.363	**10271**	**10320.363**	**11.1%**

C. Evaluation on Dynamic Model Update

We evaluate the model update performance of QTree by comparing the following alternatives: *NoTrain* (no model updates), *ReTrain* (periodic model reconstruction), and *AdaIncr* (our updating method in QTree) in Figure 5, which involves both *data updates* and *workload shifts*.

NoTrain suffers from poor accuracy because its stale model does not adapt to evolving data and workloads well. In contrast, both *ReTrain* and *AdaIncr* preserve the estimation accuracy of QTree, demonstrating their ability to capture these changes effectively. Notably, *AdaIncr* reduces update time by 30%–60% while achieving accuracy comparable to that of *ReTrain*. As the data and workload grow, the cumulative update time of *AdaIncr* increases gradually, whereas that of *ReTrain* rises sharply. This gap occurs because *ReTrain* rebuilds the entire model over increasingly large datasets, making each update more expensive. By continuously incorporating changes, *AdaIncr* processes only a small data batch and modifies only a few affected nodes each time, thereby avoiding unnecessary reconstruction of unchanged model components. This incremental strategy keeps model updates lightweight and improves scalability as the data expands with workload shifts.

D. Evaluation of Multi-Table CardEst

In this section, we evaluate multi-table CardEst and report the results on two datasets and workloads, *JOB-light* and *STATS-CEB*, in Table III. We compare QTree with *DeepDB*, *FLAT*, *NeuroCard* and *FactorJoin*.

First, QTree achieves the best average estimation accuracy across the datasets. *FactorJoin* is inaccurate on both datasets because it coarsely approximates the effects of single-table filtering on statistics of join keys. For other baselines, all methods obtain comparable Q-errors on *JOB-light* due to the simple join form and predicate patterns. On more complex *STATS-CEB*, QTree achieves the lowest mean Q-error by a significant margin. Baselines are trained on samples of pre-joined tables, reducing accuracy. This is due to the massive pre-joined tables according to the complex join forms. In contrast, QTree is join-free and less sensitive to complexity of join forms. Table IV shows that QTree remains accurate across star, chain and complex mixture join forms; *DeepDB* and *FLAT* exhibit substantially higher errors on complex mixture joins; *NeuroCard* and *FactorJoin* have high errors across all join forms.

Second, QTree performs only slightly slower inference than *FactorJoin* (the fastest but inaccurate). And QTree is $4\times$–$40\times$ faster than other baselines on *STATS-CEB* in particular. In spite of being faster than the slowest *NeuroCard*, *DeepDB* and *FLAT* require 87 and 175 ms large inference time because their SPNs over pre-joined tables with exponentially growing size contain more nodes and layers. By join-free multi-table CardEst method, QTree keeps inference cost low which exhibits linear growth with the number of joined tables.

Third, QTree offers competitive construction time and storage, avoiding model construction on large pre-joined tables. Its model is larger on *JOB-Light* because duplicated non-join-key values across table subsets enlarge leaf histograms. Replacing histograms with lightweight piecewise linear models (PLMs) could further reduce storage.

E. Evaluation on End-to-End Query Execution

Based on the multi-table CardEst results, we evaluate the effect of CardEst models on end-to-end query execution in PostgreSQL [1] using the settings described in Section VII-A. We run the end-to-end benchmark on *JOB-light* and *STATS-CEB*, comparing QTree with the PostgreSQL internal estimator [1] and learned baselines.

The results are reported in Table V. In general, learned CardEst methods improve the effectiveness of query optimization in most cases, compared with the original PostgreSQL estimator. In particular, QTree reduces end-to-end query total time by 12.5% on *JOB-light* and 11.1% on *STATS-CEB*, both superior to all baselines.

On the *JOB-light* dataset, all learned CardEst methods reduce query execution time due to their relatively small errors. And CardEst time accounts for a non-negligible fraction of

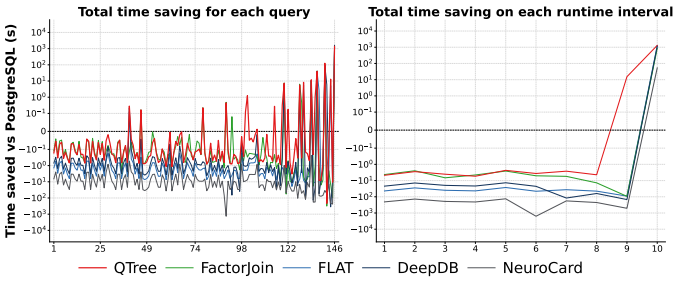


Fig. 6. Performance of STATS-CEB Queries Ordered by PostgreSQL Time.

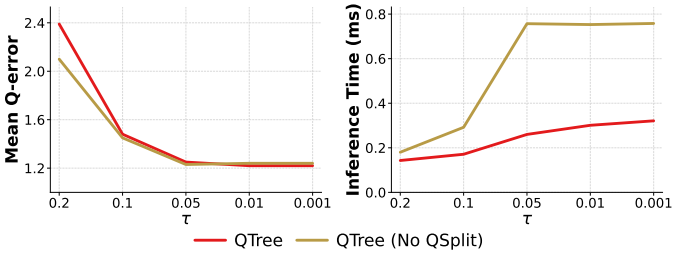


Fig. 7. Ablation on `QTree` Single-table CardEst: τ and `QSplit` On-Off.

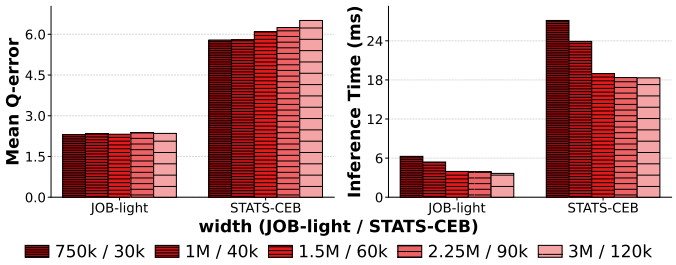


Fig. 8. Ablation on `QTree` Multi-table CardEst: *width*.

the total time. Although *NeuroCard* brings even shorter query execution time than `QTree`, it is too slow in inference to achieve overall efficiency (only 3.8% total time saved). Thus, the fast `QTree` saved the most total time (more than 10%).

On the more complex *STATS-CEB* workload, *NeuroCard* fails to optimize query execution (-28.6% total time saved) because it delivers very high CardEst errors, while other baselines and `QTree` all produce more accurate cardinalities than the PostgreSQL estimator, resulting in better join orders and access paths and lower execution time. Since `QTree` is more accurate in CardEst on *STATS-CEB*, it achieves the least query execution time on this benchmark. Meanwhile, `QTree` only takes slightly more CardEst time than the fastest but inaccurate *FactorJoin*. Thus, `QTree` is completely ahead of baselines, saving total query time more than 10%. Figure 6 orders STATS-CEB queries by increasing execution time in the original PostgreSQL and compares the total time savings of different CardEst methods. `QTree` achieves the largest savings for most queries, with its advantage widening for slower queries of the original PostgreSQL.

F. Ablation Study of `QTree`

`QTree` Single-table CardEst. We conduct an ablation study on *GAS*, the largest single-table dataset. Figure 7 shows the trade-off between estimation accuracy and inference time as we vary τ, the threshold for `QProduct`. Without `QSplit`, the workload remains largely unpartitioned, so CardEst performance changes little as τ increases beyond a tight level. And `QSplit` significantly accelerates inference (e.g., by $2.5\times$ at $\tau = 0.01$) while maintaining accurate CardEst. With an overly permissive τ, however, partitioning the workload into query subsets via `QSplit` makes `QProduct` more likely to split column sets that retain strong access affinity within these subsets, resulting in slightly larger errors.

`QTree` Multi-table CardEst. We conduct ablation studies on *JOB-light* and *STATS-CEB* by varying *width*. Figure 8 shows that smaller *width* values produce finer grids, thereby improving accuracy at the cost of longer inference time. By effectively combining join-key statistics with base-table filtering, `QTree` achieves accurate multi-table CardEst with fast inference even using coarse grids.

VIII. Related Work

Traditional CardEst Methods. Traditional methods use histograms with column independence (Postgres [1]), query execution on samples [2], [4], [5], multi-dimensional histograms (MHist [6]) on data with small domain, or probabilistic graphical models (Bayes [7], [15], [38], [39]). Their simplifying assumptions on correlations can cause large errors. And some pessimistic CardEst methods (SafeBound [40],LpBound [41],CorrBound [42]) focus on deriving tighter upper bounds on query cardinalities rather than directly minimizing CardEst errors.

Learning-based CardEst Methods. Query-driven methods learn regression models from queries to cardinalities. MSCN [8] uses four MLPs to predict selectivity from query encodings, while LW-XGB/NN [9] feeds concatenated range predicates into XGBoost [43] or a small NN. Data-driven methods model joint column distributions. Naru, UAE [10], [19] use autoregressive models (e.g., MADE [44]) and progressive sampling for CardEst, while DeepDB, FLAT [13], [14] use sum-product network models. FactorJoin [31] performs multi-table CardEst using join-key buckets, but its accuracy is limited by coarse estimates of how single-table filtering affects bucket statistics. Despite advancements in learning-based CardEst, existing methods may struggle to simultaneously optimize the three key criteria.

IX. Conclusion

In this paper, we have introduced `QTree`, a tree-structured CardEst model that integrates both data distribution and query workload. `QTree` partitions columns based on both data correlations and query access patterns, thereby improving inference efficiency and reducing model size without sacrificing estimation accuracy. We have formalized `QTree` as a tree containing two new node types: `QProduct` and `QSplit`. We have also proposed a multi-table CardEst method by extending `QTree`. We have conducted extensive experiments to evaluate `QTree`. The experimental results have demonstrated that it achieves superior and robust performance on the three key criteria, compared with state-of-the-art approaches.

X. AI-Generated Content Acknowledgement

This work used AI-assisted tools during manuscript preparation. GPT-6 (OpenAI) was used to assist with language polishing and writing plotting code for figures presenting experimental results. All AI-generated text and figures were reviewed, verified, and revised as needed by the authors, who take full responsibility for the correctness and originality of the final content.

References

[1] "1996-2024 the postgresql global development group, postgresql 12.20 documentation: Chapter 59. genetic query optimizer," *https://www.postgresql.org/docs/12/geqo-intro2.html*.

[2] P. J. Haas, J. F. Naughton, S. Seshadri, and L. Stokes, "Sampling-based estimation of the number of distinct values of an attribute," in *VLDB'95, Proceedings of 21th International Conference on Very Large Data Bases, September 11-15, 1995, Zurich, Switzerland* (U. Dayal, P. M. D. Gray, and S. Nishio, eds.), pp. 311–322, Morgan Kaufmann, 1995.

[3] M. Charikar, S. Chaudhuri, R. Motwani, and V. R. Narasayya, "Towards estimation error guarantees for distinct values," in *Proceedings of the Nineteenth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems, May 15-17, 2000, Dallas, Texas, USA* (V. Vianu and G. Gottlob, eds.), pp. 268–279, ACM, 2000.

[4] S. Chaudhuri, G. Das, and U. Srivastava, "Effective use of block-level sampling in statistics estimation," in *Proceedings of the ACM SIGMOD International Conference on Management of Data, Paris, France, June 13-18, 2004* (G. Weikum, A. C. König, and S. Deßloch, eds.), pp. 287–298, ACM, 2004.

[5] V. Leis, B. Radke, A. Gubichev, A. Kemper, and T. Neumann, "Cardinality estimation done right: Index-based join sampling," in *8th Biennial Conference on Innovative Data Systems Research, CIDR 2017, Chaminade, CA, USA, January 8-11, 2017, Online Proceedings*, www.cidrdb.org, 2017.

[6] V. Poosala and Y. E. Ioannidis, "Selectivity estimation without the attribute value independence assumption," in *VLDB'97, Proceedings of 23rd International Conference on Very Large Data Bases, August 25-29, 1997, Athens, Greece* (M. Jarke, M. J. Carey, K. R. Dittrich, F. H. Lochovsky, P. Loucopoulos, and M. A. Jeusfeld, eds.), pp. 486–495, Morgan Kaufmann, 1997.

[7] C. K. Chow and C. N. Liu, "Approximating discrete probability distributions with dependence trees," *IEEE Trans. Inf. Theory*, vol. 14, no. 3, pp. 462–467, 1968.

[8] A. Kipf, T. Kipf, B. Radke, V. Leis, P. A. Boncz, and A. Kemper, "Learned cardinalities: Estimating correlated joins with deep learning," in *9th Biennial Conference on Innovative Data Systems Research, CIDR 2019, Asilomar, CA, USA, January 13-16, 2019, Online Proceedings*, www.cidrdb.org, 2019.

[9] A. Dutt, C. Wang, A. Nazi, S. Kandula, V. R. Narasayya, and S. Chaudhuri, "Selectivity estimation for range predicates using lightweight models," *Proc. VLDB Endow.*, vol. 12, no. 9, pp. 1044–1057, 2019.

[10] Z. Yang, E. Liang, A. Kamsetty, C. Wu, Y. Duan, X. Chen, P. Abbeel, J. M. Hellerstein, S. Krishnan, and I. Stoica, "Deep unsupervised cardinality estimation," *Proc. VLDB Endow.*, vol. 13, no. 3, pp. 279–292, 2019.

[11] Z. Yang, A. Kamsetty, S. Luan, E. Liang, Y. Duan, X. Chen, and I. Stoica, "Neurocard: One cardinality estimator for all tables," *Proc. VLDB Endow.*, vol. 14, no. 1, pp. 61–73, 2020.

[12] A. Molina, A. Vergari, N. D. Mauro, S. Natarajan, F. Esposito, and K. Kersting, "Mixed sum-product networks: A deep architecture for hybrid domains," in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence, (AAAI-18), the 30th innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI-18), New Orleans, Louisiana, USA, February 2-7, 2018* (S. A. McIlraith and K. Q. Weinberger, eds.), pp. 3828–3835, AAAI Press, 2018.

[13] B. Hilprecht, A. Schmidt, M. Kulessa, A. Molina, K. Kersting, and C. Binnig, "Deepdb: Learn from data, not from queries!," *Proc. VLDB Endow.*, vol. 13, no. 7, pp. 992–1005, 2020.

[14] R. Zhu, Z. Wu, Y. Han, K. Zeng, A. Pfadler, Z. Qian, J. Zhou, and B. Cui, "FLAT: fast, lightweight and accurate method for cardinality estimation," *Proc. VLDB Endow.*, vol. 14, no. 9, pp. 1489–1502, 2021.

[15] K. Tzoumas, A. Deshpande, and C. S. Jensen, "Lightweight graphical models for selectivity estimation without independence assumptions," *Proc. VLDB Endow.*, vol. 4, no. 11, pp. 852–863, 2011.

[16] X. Wang, C. Qu, W. Wu, J. Wang, and Q. Zhou, "Are we ready for learned cardinality estimation?," *Proc. VLDB Endow.*, vol. 14, no. 9, pp. 1640–1654, 2021.

[17] G. Graefe and W. J. McKenna, "The volcano optimizer generator: Extensibility and efficient search," in *Proceedings of the Ninth International Conference on Data Engineering, April 19-23, 1993, Vienna, Austria*, pp. 209–218, IEEE Computer Society, 1993.

[18] Y. E. Ioannidis and S. Christodoulakis, "On the propagation of errors in the size of join results," in *Proceedings of the 1991 ACM SIGMOD International Conference on Management of Data, Denver, Colorado, USA, May 29-31, 1991* (J. Clifford and R. King, eds.), pp. 268–277, ACM Press, 1991.

[19] P. Wu and G. Cong, "A unified deep model of learning from both data and queries for cardinality estimation," in *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021* (G. Li, Z. Li, S. Idreos, and D. Srivastava, eds.), pp. 2009–2022, ACM, 2021.

[20] H. Poon and P. M. Domingos, "Sum-product networks: A new deep architecture," in *IEEE International Conference on Computer Vision Workshops, ICCV 2011 Workshops, Barcelona, Spain, November 6-13, 2011*, pp. 689–690, IEEE Computer Society, 2011.

[21] D. López-Paz, P. Hennig, and B. Schölkopf, "The randomized dependence coefficient," in *Advances in Neural Information Processing Systems 26: 27th Annual Conference on Neural Information Processing Systems 2013. Proceedings of a meeting held December 5-8, 2013, Lake Tahoe, Nevada, United States* (C. J. C. Burges, L. Bottou, Z. Ghahramani, and K. Q. Weinberger, eds.), pp. 1–9, 2013.

[22] M. Stoer and F. Wagner, "A simple min-cut algorithm," *J. ACM*, vol. 44, no. 4, pp. 585–591, 1997.

[23] K. Okumoto, T. Fukunaga, and H. Nagamochi, "Divide-and-conquer algorithms for partitioning hypergraphs and submodular systems," *Algorithmica*, vol. 62, no. 3-4, pp. 787–806, 2012.

[24] T. Hirayama, Y. Liu, K. Makino, K. Shi, and C. Xu, "A polynomial time algorithm for finding a minimum 4-partition of a submodular function," *Math. Program.*, vol. 207, no. 1, pp. 717–732, 2024.

[25] "QTree_TechnicalReport," *https://github.com/rucjrliu/QTreeCode*.

[26] "Uci ml repository, gas sensor array temperature modulation data set," *https://archive.ics.uci.edu/ml/datasets/Gas+sensor+array+temperature+modulation*.

[27] "wuziniu, github repository: Fspn," *https://github.com/wuziniu/FSPN*.

[28] "Uci ml repository, classification of pixels into 7 forest cover types," *https://archive.ics.uci.edu/dataset/31/covertype*.

[29] "Uci ml repository, individual household electric power consumption," *https://archive.ics.uci.edu/dataset/235/individual+household+electric+power+consumption*.

[30] "Research institute for mathematics & computer science in the netherlands, imdb.tgz," *https://homepages.cwi.nl/ boncz/job/imdb.tgz*.

[31] Z. Wu, P. Negi, M. Alizadeh, T. Kraska, and S. Madden, "Factorjoin: A new cardinality estimation framework for join queries," *Proc. ACM Manag. Data*, vol. 1, no. 1, pp. 41:1–41:27, 2023.

[32] Y. Han, Z. Wu, P. Wu, R. Zhu, J. Yang, L. W. Tan, K. Zeng, G. Cong, Y. Qin, A. Pfadler, Z. Qian, J. Zhou, J. Li, and B. Cui, "Cardinality estimation in DBMS: A comprehensive benchmark evaluation," *Proc. VLDB Endow.*, vol. 15, no. 4, pp. 752–765, 2021.

[33] "Tpc, tpc-h," *https://www.tpc.org/tpch/default5.asp*.

[34] "Tpc, tpc-ds," *https://www.tpc.org/tpcds/default5.asp*.

[35] V. Leis, A. Gubichev, A. Mirchev, P. A. Boncz, A. Kemper, and T. Neumann, "How good are query optimizers, really?," *Proc. VLDB Endow.*, vol. 9, no. 3, pp. 204–215, 2015.

[36] "Jingyi yang, github repository: pagegitss/uae," *https://github.com/pagegitss/UAE*.

[37] "Github repository: Factorjoin.," *https://github.com/SavingBigHead/FactorJoin*.

[38] A. Deshpande, M. N. Garofalakis, and R. Rastogi, "Independence is good: Dependency-based histogram synopses for high-dimensional data," in *Proceedings of the 2001 ACM SIGMOD international conference on Management of data, Santa Barbara, CA, USA, May 21-24, 2001* (S. Mehrotra and T. K. Sellis, eds.), pp. 199–210, ACM, 2001.

[39] L. Getoor, B. Taskar, and D. Koller, "Selectivity estimation using probabilistic models," in *Proceedings of the 2001 ACM SIGMOD international conference on Management of data, Santa Barbara, CA, USA, May 21-24, 2001* (S. Mehrotra and T. K. Sellis, eds.), pp. 461–472, ACM, 2001.

[40] K. B. Deeds, D. Suciu, and M. Balazinska, "Safebound: A practical system for generating cardinality bounds," *Proc. ACM Manag. Data*, vol. 1, no. 1, pp. 53:1–53:26, 2023.

[41] H. Zhang, C. Mayer, M. Abo Khamis, D. Olteanu, and D. Suciu, "Lpbound: Pessimistic cardinality estimation using ℓp-norms of degree sequences," *Proceedings of the ACM on Management of Data*, vol. 3, no. 3, pp. 1–27, 2025.

[42] C. Mayer, H. Zhang, M. A. Khamis, K. Deeds, D. Olteanu, and D. Suciu, "Corrbound: Cardinality estimation accounting for inter- and intra-relation correlations," *Proc. ACM Manag. Data*, vol. 4, no. 1, pp. 19:1–19:27, 2026.

[43] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13-17, 2016* (B. Krishnapuram, M. Shah, A. J. Smola, C. C. Aggarwal, D. Shen, and R. Rastogi, eds.), pp. 785–794, ACM, 2016.

[44] M. Germain, K. Gregor, I. Murray, and H. Larochelle, "MADE: masked autoencoder for distribution estimation," in *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015* (F. R. Bach and D. M. Blei, eds.), vol. 37 of *JMLR Workshop and Conference Proceedings*, pp. 881–889, JMLR.org, 2015.

APPENDIX

A. PROOF of LEMMA 1

Lemma 1. *The problem of* `QProduct` *construction is equivalent to the minimum k-cut problem.*

Proof. Recall that the minimum k-cut problem is defined as follows: Given an undirected graph $G = (V, E)$ with edge weights $w : E \to \mathbb{R}$ and an integer k, the goal is to partition the vertex set V into k disjoint subsets $C_1, C_2, \ldots, C_k$ such that the following objective is minimized: $\sum_{i=1}^{k-1} \sum_{j=i+1}^{k} \sum_{v_1 \in C_i, v_2 \in C_i} w(\{v_1, v_2\})$, which corresponds to the sum of weights of edges crossing different partitions.

In the context of `QProduct` construction, the column set A and the access affinity $\texttt{AFF}(a_i, a_j)$ between two columns can be naturally mapped to the vertex set V and edge weights $w(v_1, v_2)$ in the minimum k-cut problem, respectively. Moreover, the objective of minimizing the inter-partition affinity (IPA) in `QProduct` construction is equivalent to the objective of the minimum k-cut problem. Thus, we prove the lemma.

We prove that the `QProduct` construction problem can be reduced from K-CUT problem. Given an arbitrary undirected graph $G = (V, E)$ and parameter K, we make a column set $A = \{a_1, \ldots, a_{|V|}\}$ corresponding to V and we set $\texttt{AFF}(a_i, a_j|Q) = \texttt{AFF}(a_j, a_i|Q) = w_l / \max\{w|e = (u, v, w) \in E\}$ for each edge $e_l = (v_i, v_j, w_l)$. Note that the `AFF` matrix we set must be legal since the assumed query workload Q which contains $\texttt{AFF}(a_i, a_j|Q) * \max\{w|e = (u, v, w) \in E\}$ queries with predicates only on column a_i and a_j for each $\texttt{AFF}(a_i, a_j|Q)$ value is easy to generate. Then we solve `QProduct` construction problem to partition A into K subsets: $A = \{A_1, \ldots, A_K\}$ and we get the answer of the K-CUT problem: minimized $IPA(A|Q)$ multiplying $\max\{w|e = (u, v, w) \in E\}$.

So far, there is no efficient deterministic algorithm for K-CUT problem. When $K = 2$, it can be solved by an $O(|A|^3)$ but actually slow MIN-CUT algorithm [22]. When $K = 3$, it can be solved by an $O(|A|^3\tau(|A|))$ 3-CUT algorithm [23] where $\tau(|A|)$ is the cost of computing goal function which is $O(|A|^2)$ under the background of our `QProduct` construction problem *i.e.,* the algorithm costs $O(|A|^5)$. When $K = 4$, it can be solved by an $O(|A|^6\tau(|A|))$ *i.e.,* $O(|A|^8)$ 4-CUT algorithm [24]. When $K \geq 5$, there is no polynomial time algorithm found so far while the research of 4-CUT algorithm [24] made a corollary that the 5-CUT algorithm should cost $O(|A|^{14}\tau(|A|))$ *i.e.,* $O(|A|^{16})$. □

B. PROOF of LEMMA 2

Lemma 2. *For* `QSplit`, $\overline{\texttt{IPA}}(\mathcal{A}|Q)$ *provides an upper estimate of the optimal result* $\texttt{IPA}(\mathcal{A}^*|Q)$*, given query set Q.*

Proof. Due to the space constraints, we provide only a proof sketch. First, we show that $\texttt{IPA}(\mathcal{A}^*|Q)$ can be decomposed into the IPA scores of distinct query patterns, *i.e.,* $\texttt{IPA}(\mathcal{A}^*|Q) = \sum_{p_i \in Q} \texttt{IPA}(\mathcal{A}^*|\{p_i\})$. Next, we prove that for any query pattern $p_i \in Q$, its optimal IPA score $\texttt{IPA}(\mathcal{A}^*|\{p_i\})$ is bounded by the upper term $n_i \cdot z_{ij} \cdot (\|p_i\| - z_{ij})$, where p_j is any query pattern satisfying $z_{ij} > 0$. To see why this holds, note that z_{ij} represents the number of shared accessed columns between patterns p_i and p_j. The term $(\|p_i\| - z_{ij})$ captures the remaining accessed columns in p_i that are not shared with p_j. Thus, $n_i \cdot z_{ij} \cdot (\|p_i\| - z_{ij})$ is the IPA score of the specific partitioning scheme as illustrated in Figure 2(a), which is larger than the optimal result $\texttt{IPA}(\mathcal{A}^*|\{p_i\})$. Moreover, if $z_{ij} = 0$ for all $j \neq i$, meaning that p_i shares no common accessed columns with any other query patterns, then the optimal column partition for p_i incurs no IPA cost, implying $\texttt{IPA}(\mathcal{A}^*|\{p_i\}) = 0$. Thus, we conclude that $\texttt{IPA}(\mathcal{A}^*|Q) \leq \overline{\texttt{IPA}}(\mathcal{A}|Q)$, proving the lemma. □

C. PROOF of LEMMA 3

Lemma 3. *For* `QSplit`*, the problem of partitioning Q into $\mathcal{Q} = \{Q_1, Q_2, \ldots, Q_n\}$ to minimize the upper bound of inter-partition affinity,* i.e., $\sum_{k=1}^{n} \overline{IPA}(\mathcal{A}_k|Q_k)$*, is reduced from the NP-hard Max-Cut problem.*

Proof. To convert an arbitrary graph $G(V, E)$ to `QSplit` problem, we generate $\mathbf{q_i}$ corresponding to each $v_i \in V$ and we generate $w = |\mathbf{q_i}|z_{i,j}(x_i - z_{i,j}) + |\mathbf{q_j}|z_{i,j}(x_j - z_{i,j})$ for each $e_{ij} \in E$. To maximize the sum of edge weights that are cut off in G, the sum of edge weights remained in all components is minimized, which is equivalent to minimizing $\texttt{SIQ}(\mathcal{Q}|A)$ after converting G to `QProduct` problem. Thus, minimizing $\texttt{SIQ}(\mathcal{Q}|A)$ can be reduced to MAX-CUT problem. □

D. Complexity Analysis of Offline `QTree` Construction

Given data T, workload Q and column set $|\mathcal{A}| = m$, supposing a balanced c-way `QTree` tree with n nodes (s `Sum` nodes, x `QSplit` nodes, u `Product` — `QProduct` nodes, less than and around n `Leaf` nodes) and z sample size for RDC. For each `Leaf` node, the number of tuples is around $|T|/s$ so its complexity is $O(n|T|/s)$. For each of the data tuples, it will go through $log_c s$ `Sum` nodes and cost $O(cm)$ in each node so the complexity of `Sum` nodes is $O(cm|T|log_c s)$. Similarly, the complexity of `QSplit` nodes is $O(cm|Q|log_c t)$. The cases are different with `Product` and `QProduct`, since their cost contains RDC and `AFF` matrixes computation. For RDC matrix computation, the cost is constant $O(zlogz)$. So the complexity of `Product` nodes is $O(nm^2 zlogz)$. For `AFF` matrix calculation, the cost is $O(m^2|Q|)$. So the complexity of `QProduct` nodes is $O((s + x + u)m^2|Q|)$. In total, the complexity of `QTree` construction is $O(m^2[nzlogz + (s+x+u)|Q|] + cm(|T|log_c s + |Q|log_c x) + n|T|/s)$.

E. Complexity Analysis of Online `QTree` Inference

Given a query q with selectivity (probability) p and constraints on m columns, suppose the `QTree` model is a balanced c-way tree with N nodes (s `Sum` nodes, x `QSplit` nodes, and u `Product`/`QProduct` nodes). It is known that the cost of a `Leaf` node with a histogram is $O(1)$, the cost of a `QSplit` node is $O(m^2 c)$, and the cost of other middle nodes is $O(mc)$. Thus, the worst-case complexity of the bottom-up inference method is $O(xm^2 c + (N - x)mc)$. Since there are usually a small number of `QSplit` nodes in a `QTree` model

for query routing, the time cost of our fast inference method is approximately $\max(p, \frac{1}{s}) \cdot \frac{m}{u} \cdot Nmc$.

F. Offline QTree Construction Algorithm

Algorithm 1 **ConstructQTree** (A, T, Q)

Input: A: A Column Set; T: A Table; Q: A Query Workload
Output: n: A QTree Node

Initialize a QTree node $\mathtt{n} = (A, T, Q, O)$
if $|A| = 1$ **then**
  $\mathtt{n}.O \leftarrow$ Leaf
  $\mathtt{n}.Pr_T(A) \leftarrow$ **BuildHist** (A, T)
else if $\mathcal{A} \leftarrow$ **PartitionByRDC** (A, T) is *non-singleton* **then**
  $\mathtt{n}.O \leftarrow$ Product
  $\mathtt{n.child}_i \leftarrow$ **ConstructQTree** (A_i, T, Q) for $\forall A_i \in \mathcal{A}$
else if $\mathcal{A} \leftarrow$ **PartitionByAFF** (A, Q) is *non-singleton* **then**
  $\mathtt{n}.O \leftarrow$ Product
  $\mathtt{n.child}_i \leftarrow$ **ConstructQTree** (A_i, T, Q) for $\forall A_i \in \mathcal{A}$
else if $\mathcal{Q} \leftarrow$ **SplitWorkload** (Q, A) is *non-singleton* **then**
  $\mathtt{n}.O \leftarrow$ QSplit
  $\mathtt{n.child}_i \leftarrow$ **ConstructQTree** (A, T, Q_i) for each $Q_i \in \mathcal{Q}$
else
  $\mathtt{n}.O \leftarrow$ Sum
  $\mathcal{T} \leftarrow$ **ClusterTable** (T, A)
  Compute weight $w_i \leftarrow |T_i|/|T|$ for each sub-table $T_i \in \mathcal{T}$
  $\mathtt{n.child}_i \leftarrow$ **ConstructQTree** (A, T_i, Q) for each $T_i \in \mathcal{T}$
end if
return node n

Algorithm 1.1 **PartitionByAFF** (Q, A)

Input: Q: A Query Workload; A: A Column Set
Output: $\mathcal{A} = \{A_1, \ldots, A_m\}$: A Collection of Column Sets

for any pair (a_i, a_j) from A **do**
  Compute $\mathtt{AFF}(a_i, a_j|Q)$ for a_i and a_j
end for
Construct a graph $G = (A, E)$ where vertices A represent columns, and an edge $e_{ij} \in E$ exists between columns a_i and a_j if their affinity $\mathtt{AFF}(a_i, a_j|Q)$ is larger than a threshold τ.
$\mathcal{G} \leftarrow$ **ConnectedComponents** (G)
return $\mathcal{A} = \{A_i \mid$ vertex set in each component $G_i \in \mathcal{G}\}$

We show our top-down **ConstructQTree** (Algorithm 1) corresponding to Section IV including its sub function **PartitionByAFF** (Algorithm 1.1) which is explained in Section IV-A and sub function **SplitWorkload** (Algorithm 1.2) which is explained in Section IV-B. In **ConstructQTree** algorithm, each node n to construct are tried different type $\mathtt{n}.O$ in the heuristic order: Leaf, Product, QProduct, QSplit, Sum for a accuracy-efficiency balanced QTree model. And the specific node construction are executed in sub functions.

Algorithm 2 **QTree-Online** $(\mathtt{n}, q)$

Input: n: A QTree Node; q: A Query
Output: $P_\mathtt{n}$: Probability Estimated at Node n

if $\mathtt{n}.O =$ Leaf **then**
  return $\mathtt{n}.P_T(q)$ based on the histogram at leaf n
else if $\mathtt{n}.O =$ Product | QProduct **then**
  $P_i \leftarrow$ **QTree-Online** $(\mathtt{n.child}_i, q)$ for each child of n with the columns satisfying $q.A \cap \mathtt{n.child}_i.A \neq \emptyset$
  return $\prod_i P_i$ from all child nodes of n
else if $\mathtt{n}.O =$ QSplit **then**
  $k \leftarrow$ **RouteQuery** $(q, \{\mathtt{n.child}_i\})$
  return **QTree-Online** $(\mathtt{n.child}_k, q)$
else
  $\mathcal{C} \leftarrow$ **SelectChildren** $(\{\mathtt{n.child}_i\}, q)$
  $P_i \leftarrow$ **QTree-Online** $(\mathtt{child}, q)$ for each $\mathtt{child} \in \mathcal{C}$
  return weighted summation $\sum_i w_i \cdot P_i$
end if

G. Online QTree Inference Algorithm

We show our top-down **QTree-Online** (Algorithm 2) based on query routing and pruning rules whose sub function **RouteQuery** is explained in Section V-A. Benefiting from sufficient optimization utilizing the features of QTree, the **QTree-Online** algorithm provides accurate and fast CardEst in online phase using lightweight QTree model which are constructed both on data and workload before.

H. Online QTree Update Algorithm

We show our efficient top-down **QTree-Update** (Algorithm 3) based on node examining whose sub function **ColumnPartitionCheck**, **WorkloadSplitCheck** and **DataPartitionCheck** are explained in Section V-B. The **QTree-Update** algorithm achieves adaptive incremental QTree update to keep the model accurate against data update and query workload shift in online phase of dynamic scenes, which is manual-triggering-free and brings little extra cost.

Algorithm 3 **QTree-Update** $(\mathtt{n}, \Delta T, \Delta Q)$

Input: n: A QTree Node, ΔT: Data Update subset, ΔQ: New Queries subset
Output: Updated QTree node $\mathtt{n}'$

$U \leftarrow False$
if $\mathtt{n}.O = \mathtt{Leaf}$ **then**
  $\mathtt{n}' \leftarrow \mathtt{n}$
  Modify Histogram at $\mathtt{n}'$ by ΔT
  return $\mathtt{n}'$
else if $\mathtt{n}.O = \mathtt{Product} \mid \mathtt{QProduct}$ **then**
  $U, \mathtt{n}' \leftarrow$**ColumnPartitionCheck** $(\mathtt{n}, \Delta T, \Delta Q)$
else if $\mathtt{n}.O = \mathtt{QSplit}$ **then**
  $U, \mathtt{n}' \leftarrow$**WorkloadSplitCheck** $(\mathtt{n}, \Delta Q)$
else if $\mathtt{n}.O = \mathtt{Sum}$ **then**
  $U, \mathtt{n}' \leftarrow$**DataPartitionCheck** $(\mathtt{n}, \Delta T)$
end if
if U **then**
  $\mathtt{n}' \leftarrow$ **ConstrcutQTree** $(\mathtt{n}'.A, \mathtt{n}'.T \cup \Delta T, \mathtt{n}'.Q \cup \Delta Q)$
else
  $\mathtt{n}'.\mathtt{child}_i \leftarrow$ **QTree-Update** $(\mathtt{n}'.\mathtt{child}_i, \Delta T_i, \Delta Q_i)$ for each child of $\mathtt{n}'$
end if
return $\mathtt{n}'$

Algorithm 1.2 **SplitWorkload** (Q, A)

Input: Q: A Query Workload; A: A Column Set
Output: $\mathcal{Q} = \{Q_1, \ldots Q_m\}$: A Collection of Sub-workloads

$\mathbf{q_1}, \ldots, \mathbf{q_w} \leftarrow$ Merge Queries q sharing the same $\mathbf{acc}_q$
$\mathtt{sim} \leftarrow$ **ComputeSim** (**ACC**(Q, A)) for each (q_i, q_j) from Q
Construct a graph $G = (V, E)$ where vertice V represents **ACC**$(Q, A)_1, \ldots,$ **ACC**$(Q, A)_{|\mathbf{ACC}(Q,A)|}$ and $e_{kl} \in E$ represents $\mathtt{sim}($**ACC**$(Q, A)_k,$ **ACC**$(Q, A)_l)$
Cut off some biggests edges of G
Sort $v \in V$ by weighted degrees in desc order
Create m empty set: $R_1, \ldots, R_m$
for $1 \leq i \leq |$**ACC**$(Q, A)|$ **do**
  Put **ACC**$(Q, A)_i$ into the subset R_j with $\min_j(\sum_{\mathbf{ACC}(Q,A)_i \in R_j} \mathtt{sim}(i,$ **ACC**$(Q, A)_k))$
end for
Create m empty query subsets: $Q_1, \ldots, Q_m$
for $q \in Q$ **do**
  Put q into Q_k if $\exists$**ACC**$(Q, A)_l \in R_k$ and q obeys **ACC**$(Q, A)_l$
end for
$\mathcal{Q} \leftarrow \{Q_1, \ldots, Q_m\}$
return $\mathcal{Q}$